

A large, dark blue, curved graphic element that spans the width of the page and curves upwards at the bottom, resembling a stylized wave or a protective shield.

PIF

PIF
LAURA GLODE

LPEM | Glola041@school.lu

Table of Contents

Lab.....	4
L1 – Raspberry Pi	4
Install “Raspberry Pi OS (64-bit)” on the microSD card.....	4
Configuration of the operating system.....	5
Enable the VNC Server too access via RVNC.	8
Configuration of a static IP address (Ethernet) and a host name (IAM code).	11
Enable the I ² C interface.....	12
Docker installation	13
Screen orientation and keyboard	14
Copy the supplied files (firmware including configuration file, website, and scripts for testing the sensors) to the Raspberry Pi.	15
How to create a docker container	17
Display the website in the kisok mode.....	20
L2 – Database.....	22
Conceptual model (MCD/CDM).....	22
Logical model (MLD / LDM).....	23
Database – Apex syntax.....	24
Dataset – Apex syntax (IA creation)	26
Database – Phpmyadmin syntax	28
Dataset – Phpmyadmin syntax.....	30
Snapshot of the test sample in Phpmyadmin	32
L4 - Web database server.....	35
Virtual or Physical Server?	35
Installation steps	36
<i>Install everything in one single command using:</i>	37
Installation of a database and website – Share Folder Method.....	38
The rights to the website files should be set so that unauthorized persons cannot access them....	39
User and Database Creation in MariaDB.....	39
The website can be successfully accessed with a browser.....	41
MySQL should be secured with mysql_secure_installation	44
All installed packages and the server's operating system should be up to date.....	46
User Story	47
L5 – Data Transfer	48
PHP Script for Data Transfer	48
Manual Data Entry Form for Measurement.....	49

Installation of the PHP Data Transfer Receiver Script on the Server	52
Station Configuration for Data Transfer	53
Errors	54
L6 - Backup Server	55
Objective	55
Backup server → connects to database server → creates and stores backup locally	55
Backup server installation	55
Installing MySQL client and FTP software.	56
Creating a MySQL backup user on the web/database server	56
Creating a backup directory	56
Creating and testing the backup script.....	57
Configure the shell script to run daily at 8:00 AM.	58
Set up the required FTP access	59
Atelier.....	60
A1 – PVC holder	60
Step one: pin pointing	60
Step two : Drilling	63
Step 3: Cleaning up	65
Step four: Assembling	67
A2 – Printed circuit board	69
List of material	69
Step 1: Verify Resistors	71
Step 2: Prepare for Soldering	71
Step 3: Solder Resistors	72
Step 4: Solder the Transistor	72
Step 5: Solder Terminal Blocks.....	73
Step 6: Solder Pin and Socket Strips.....	74
Step 7: Connect ADS1115 ADC Module	75
Step 8: Mount the circuit board on the PVC plate.	76
Overall view	77
Mistakes	78
A3 – Power Supply	79
List of material	79
Information:.....	81
Step one: Cutting and Stripping the Insulation.....	81
Step two: Preparing and Soldering the Wires	83

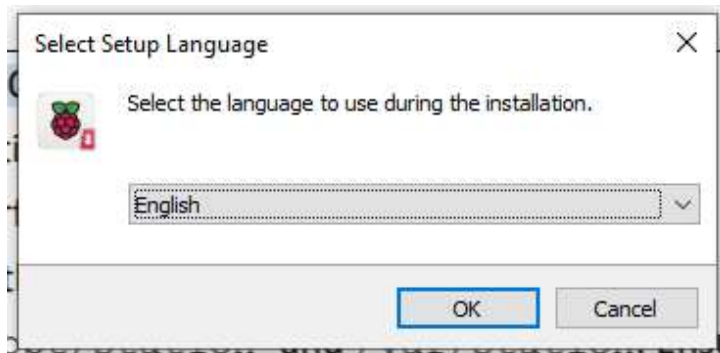
Step three: Insulating	84
A4 – Sensor #1	85
List of material	85
Step one: Connect ADS1115 ADC Module.....	86
Step two: Connect Gas Sensor	86
Step three: Raspberry configuration	87
A5 – Sensor #2.....	88
List of material	88
Step one: Sensor Installation and Cabling.....	90
Step two: Raspberry configuration	91
A6 - Assembly	95
List of material	95
Step one: Drilling the hole for the push button.....	97
Step two: Rear panel preparation drilling sensor mounting	97
Step three: Assembling the frame and base	98
Step four: Installing the push button	99
Step five: Final assembly	100
Difficulties	100
A7 - Lighting	101
List of material	101
Step one: Drilling	102
Step two: Preparing and soldering the wires	102
Step three: Installing the LED strip	103
Error:	103

Lab

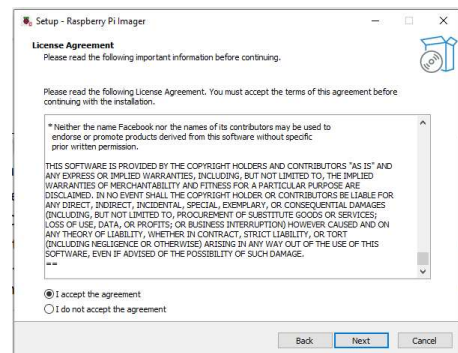
L1 – Raspberry Pi

Install “Raspberry Pi OS (64-bit)” on the microSD card

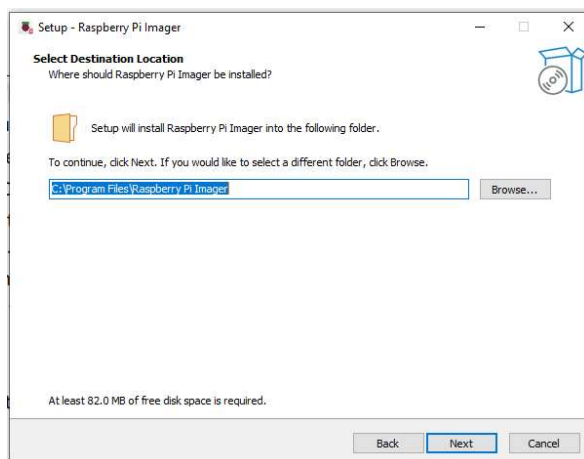
Select your preferred language.



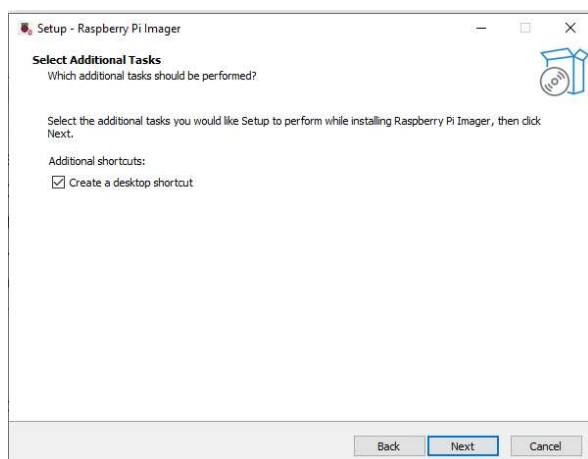
Accept the agreement



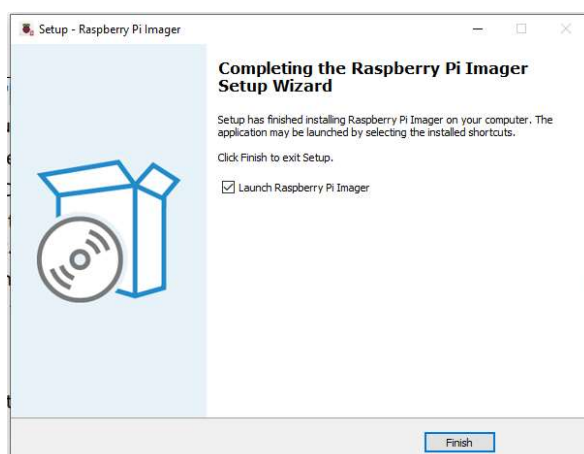
Leave the Destination Location as it is.



Select to create a desktop shortcut.

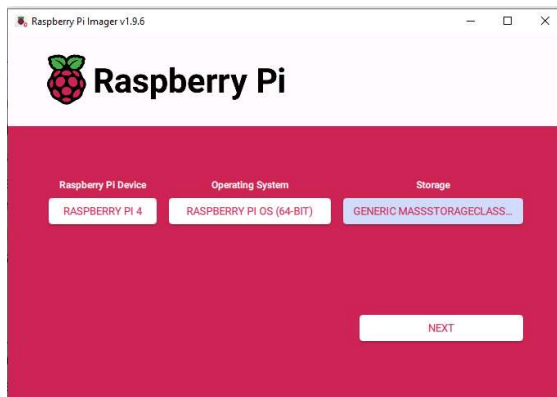


Lastly complet the setup.

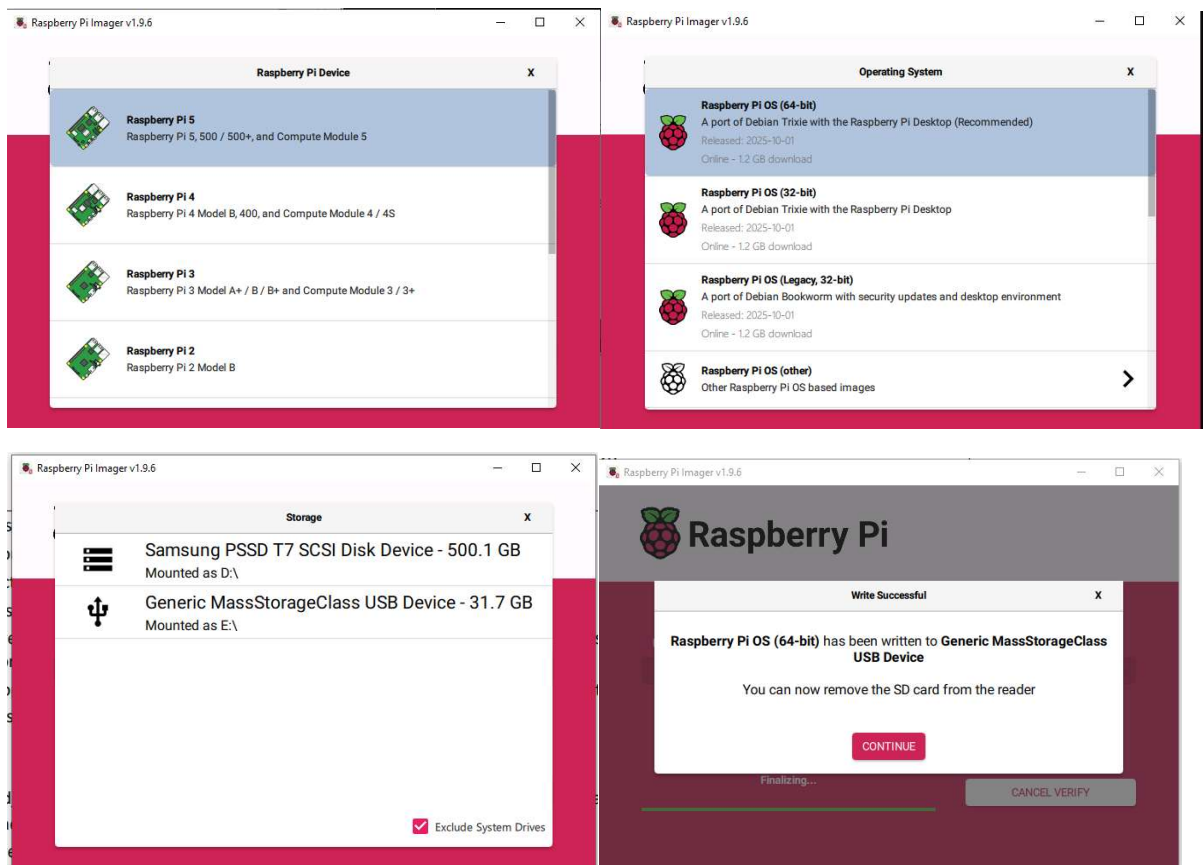


Configuration of the operating system

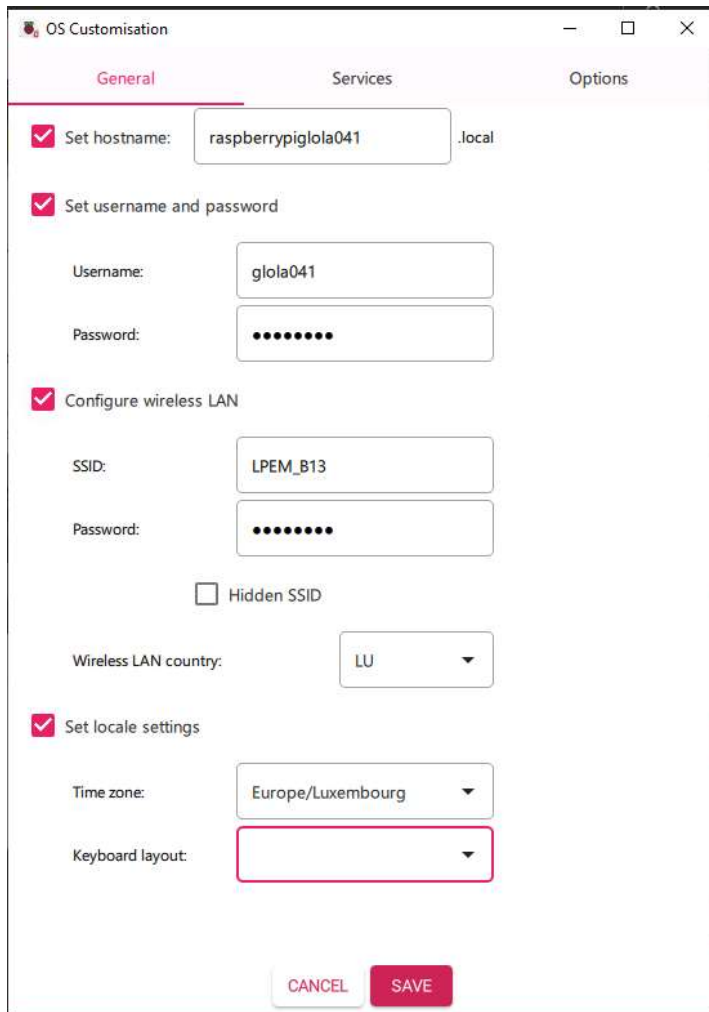
Configure the operating system according to specifications.



Specify Raspberry Pi 4 model B for the Raspberry Pi device. In the operating System select Raspberry PI OS (64-bit). Select your the microSD card for the storage, in my case it's my E\.



In the General customisation set your chosen hostname, username and password. Configure the wireless LAN and locale settings but leave the keyboard layout, it will configure itself.

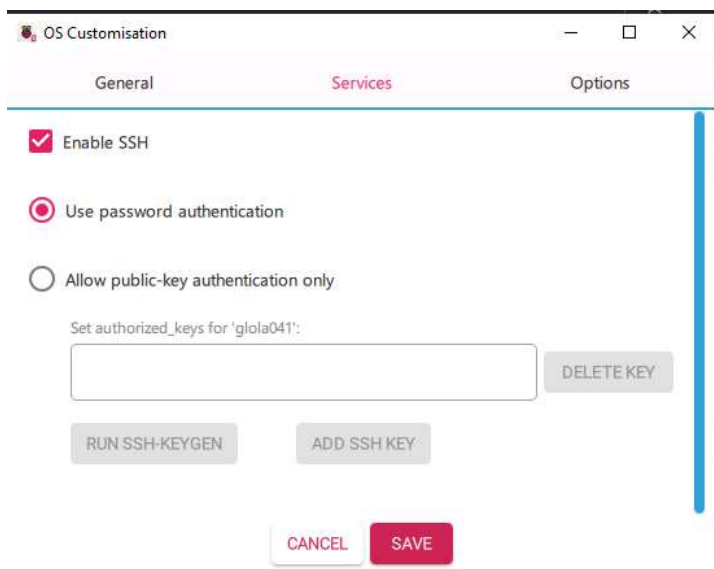


The image shows the 'OS Customisation' window with the 'General' tab selected. The window has three tabs: 'General', 'Services', and 'Options'. Under 'General', there are several settings:

- ☒ Set hostname: raspberrypiglola041.local
- ☒ Set username and password
 - Username: glola041
 - Password: [masked]
- ☒ Configure wireless LAN
 - SSID: LPEM_B13
 - Password: [masked]
 - ☐ Hidden SSID
 - Wireless LAN country: LU
- ☒ Set locale settings
 - Time zone: Europe/Luxembourg
 - Keyboard layout: [empty dropdown]

At the bottom, there are 'CANCEL' and 'SAVE' buttons.

In the Service customisation enable SSH to allow secure command-line access over the network.

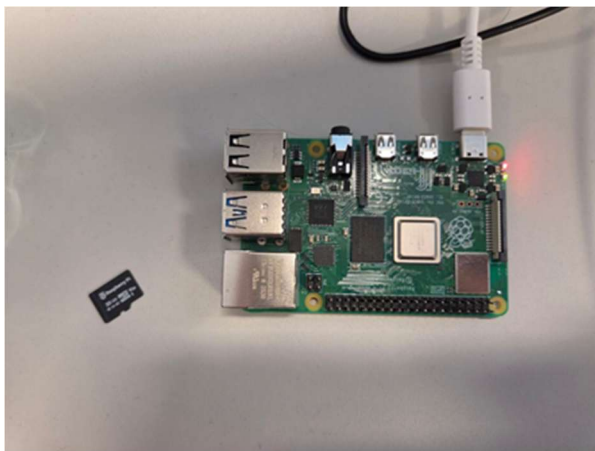
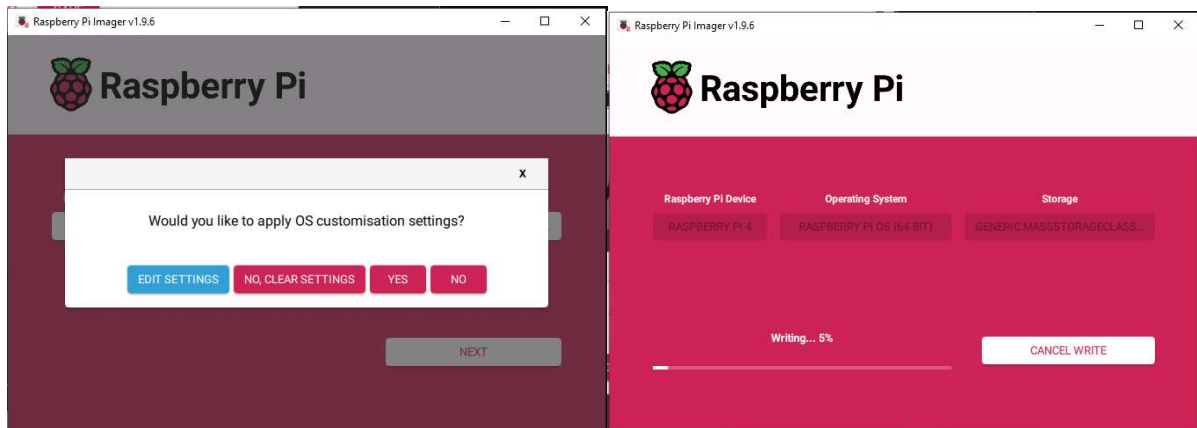


The image shows the 'OS Customisation' window with the 'Services' tab selected. The window has three tabs: 'General', 'Services', and 'Options'. Under 'Services', there are several settings:

- ☒ Enable SSH
- ☒ Use password authentication
- ☐ Allow public-key authentication only
- Set authorized_keys for 'glola041':
 - [empty text box]
 - DELETE KEY
- RUN SSH-KEYGEN
- ADD SSH KEY

At the bottom, there are 'CANCEL' and 'SAVE' buttons.

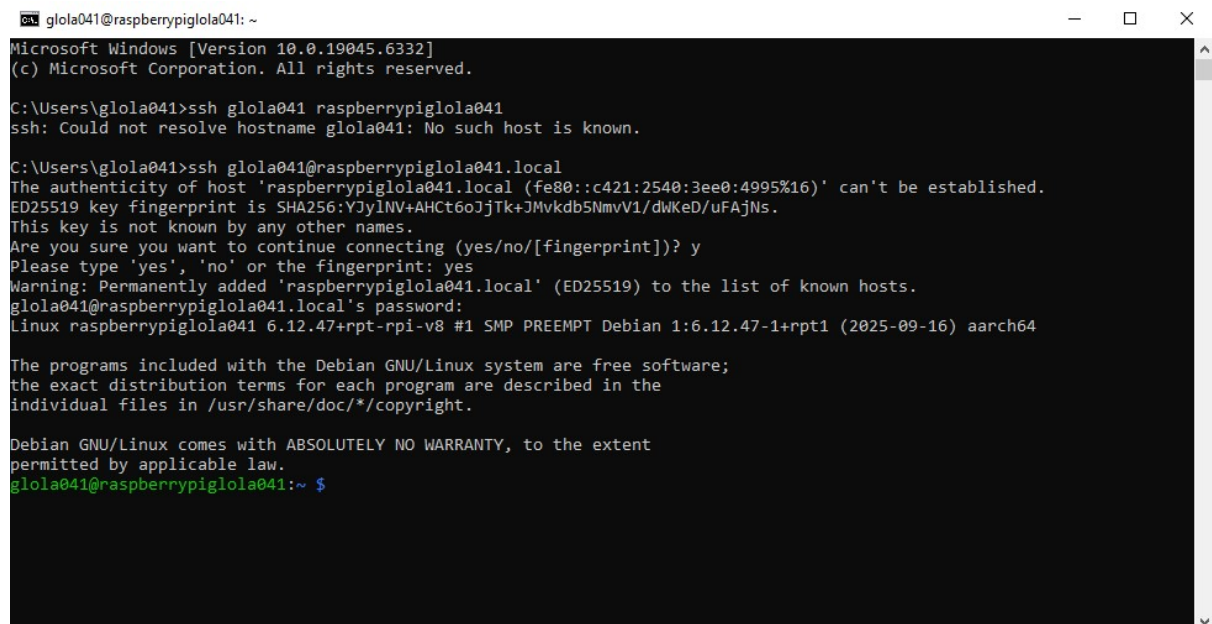
Apply the OS customisation settings and then insert the microSD card in the Raspberry Pi.



Enable the VNC Server too access via RVNC.

Open your command prompt on your desktop then enter:

➔ `ssh [username]@[ hostname].local`



```
glola041@raspberrypiglola041: ~
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

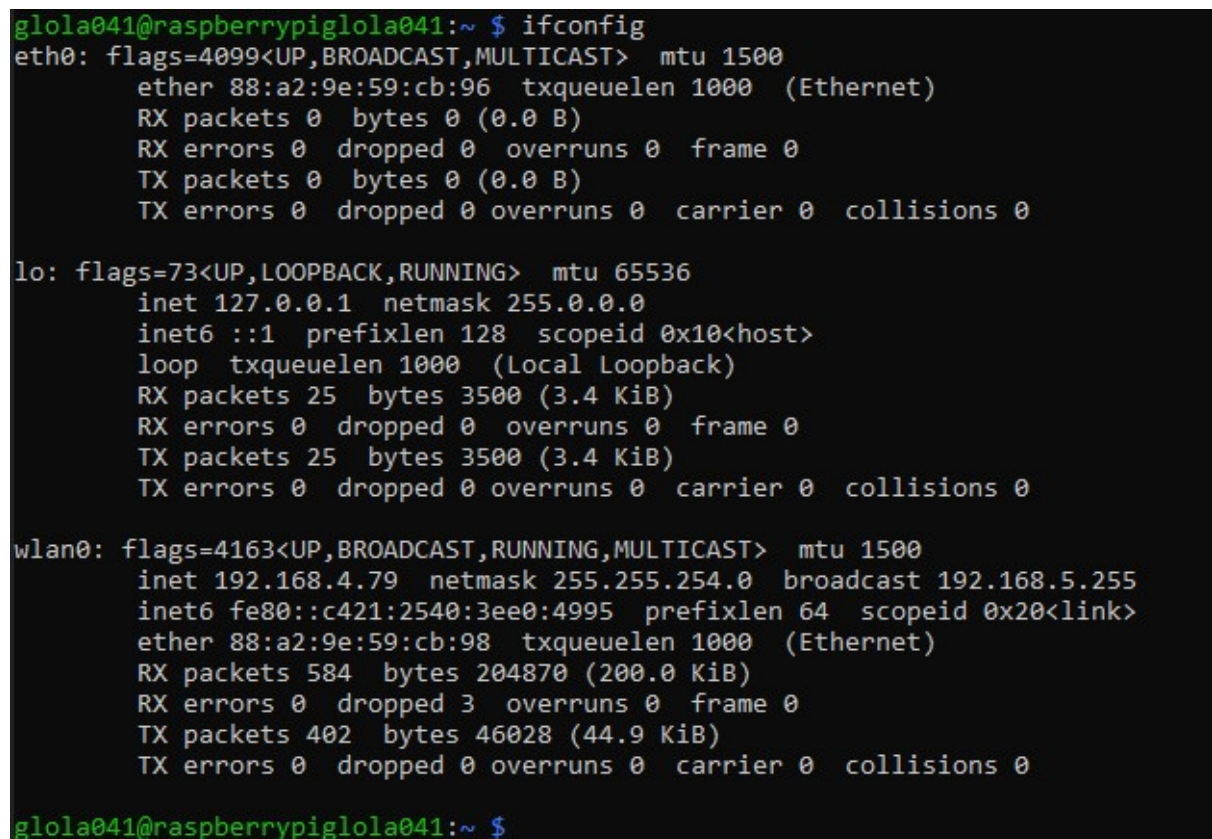
C:\Users\glola041>ssh glola041 raspberrypiglola041
ssh: Could not resolve hostname glola041: No such host is known.

C:\Users\glola041>ssh glola041@raspberrypiglola041.local
The authenticity of host 'raspberrypiglola041.local (fe80::c421:2540:3ee0:4995%16)' can't be established.
ED25519 key fingerprint is SHA256:YJy1NV+AHCT6oJjTk+JMvKdb5NmV1/dwKED/uFAjNs.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added 'raspberrypiglola041.local' (ED25519) to the list of known hosts.
glola041@raspberrypiglola041.local's password:
Linux raspberrypiglola041 6.12.47+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.12.47-1+rpt1 (2025-09-16) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
glola041@raspberrypiglola041:~ $
```

Look up your ip address in the wlan0 section, to later one use it to connect your remote desktop app of your choice.



```
glola041@raspberrypiglola041:~ $ ifconfig
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 88:a2:9e:59:cb:96 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 25 bytes 3500 (3.4 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 25 bytes 3500 (3.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.4.79 netmask 255.255.254.0 broadcast 192.168.5.255
    inet6 fe80::c421:2540:3ee0:4995 prefixlen 64 scopeid 0x20<link>
    ether 88:a2:9e:59:cb:98 txqueuelen 1000 (Ethernet)
    RX packets 584 bytes 204870 (200.0 KiB)
    RX errors 0 dropped 3 overruns 0 frame 0
    TX packets 402 bytes 46028 (44.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

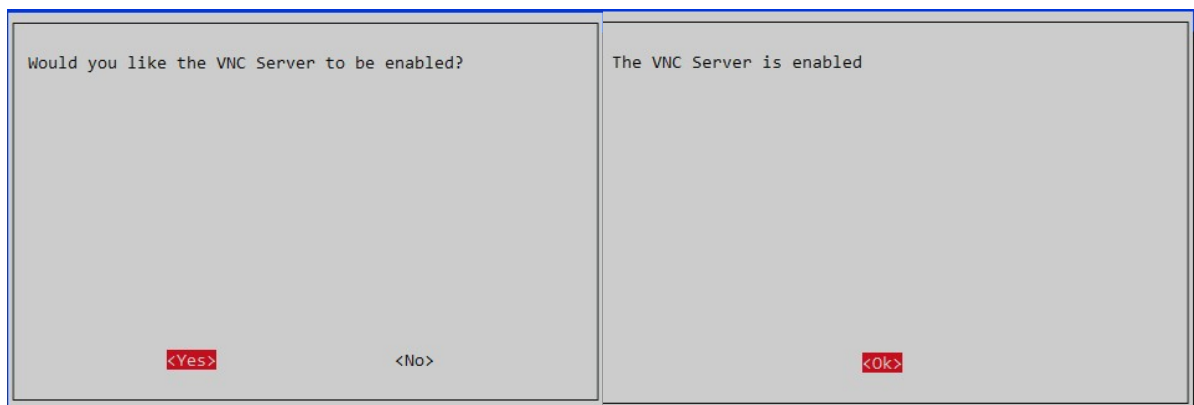
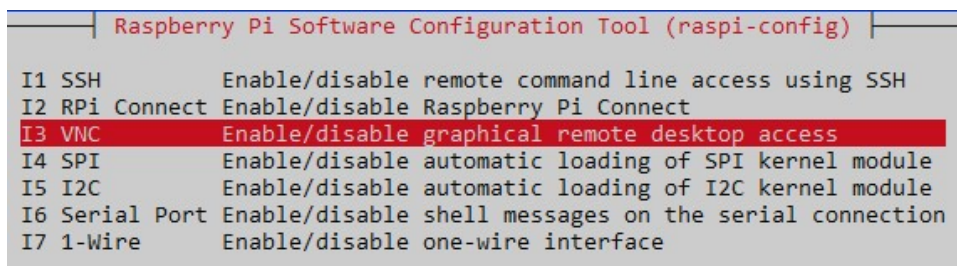
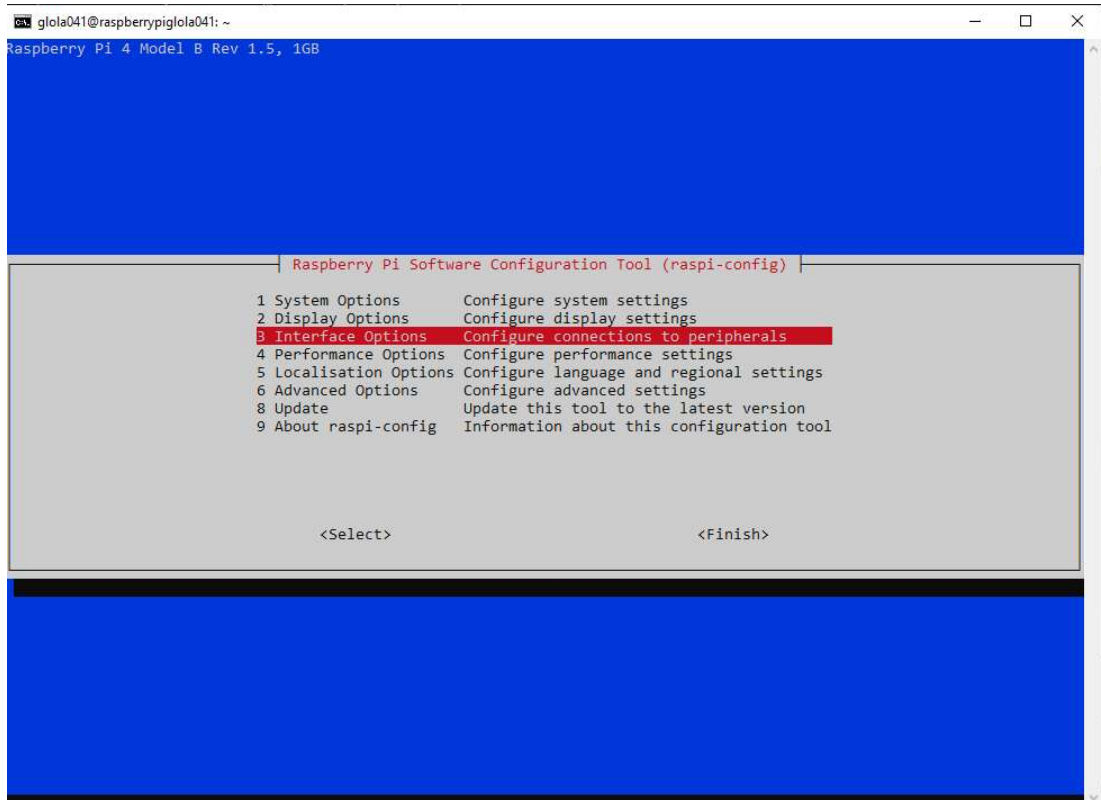
glola041@raspberrypiglola041:~ $
```

Access your Raspberry Pi Software configuration tool to enable the VNC using this command:

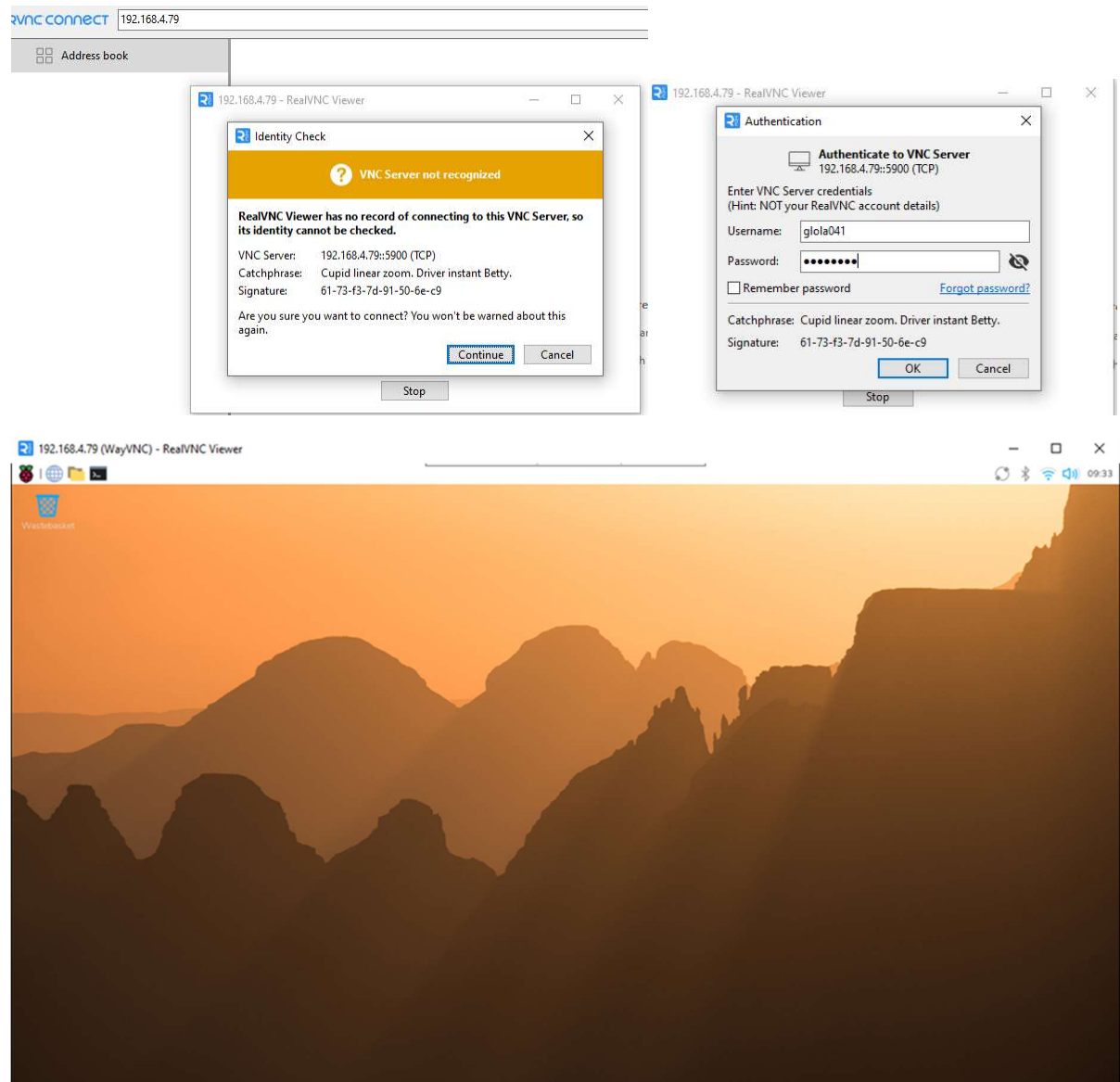
➔ `sudo raspi-config`

```
glola041@raspberrypiglola041:~ $ sudo raspi-config
Created symlink '/etc/systemd/system/multi-user.target.wants/wayvnc.service' → '/usr/lib/systemd/system/wayvnc.service'.
glola041@raspberrypiglola041:~ $
```

Go to Interface Options and enable I3 VNC:



In your remote desktop app of your choice, for example RVNC, enter the ip you previously looked up in the wlan0 section. Wenn the Identity Check-ups up click continue.



Configuration of a static IP address (Ethernet) and a host name (IAM code).

In the raspberry terminal enter:

→ `sudo nano /etc/dhcpd.conf`

```
glola041@raspberrypiglola041:~ $ sudo nano /etc/dhcpd.conf
glola041@raspberrypiglola041:~ $
```

Enter in this one for the WIFI (wlan0):

→ `interface wlan0`
`static ip_address= [Of your choice]`
`static routers= [Of your choice]`

```
GNU nano 8.4 /etc/dhcpd.conf *
# Respect the network MTU. This is applied to DHCP routes.
option interface_mtu

# Request a hostname from the network
option host_name

# Most distributions have NTP support.
#option ntp_servers

# A ServerID is required by RFC2131.
require dhcp_server_identifier

# Generate SLAAC address using the Hardware Address of the interface
#slaac hwaddr
# OR generate Stable Private IPv6 Addresses based from the DUID
slaac private

interface wlan0
static ip_address=192.168.4.209/24
static routers=192.168.4.1
```

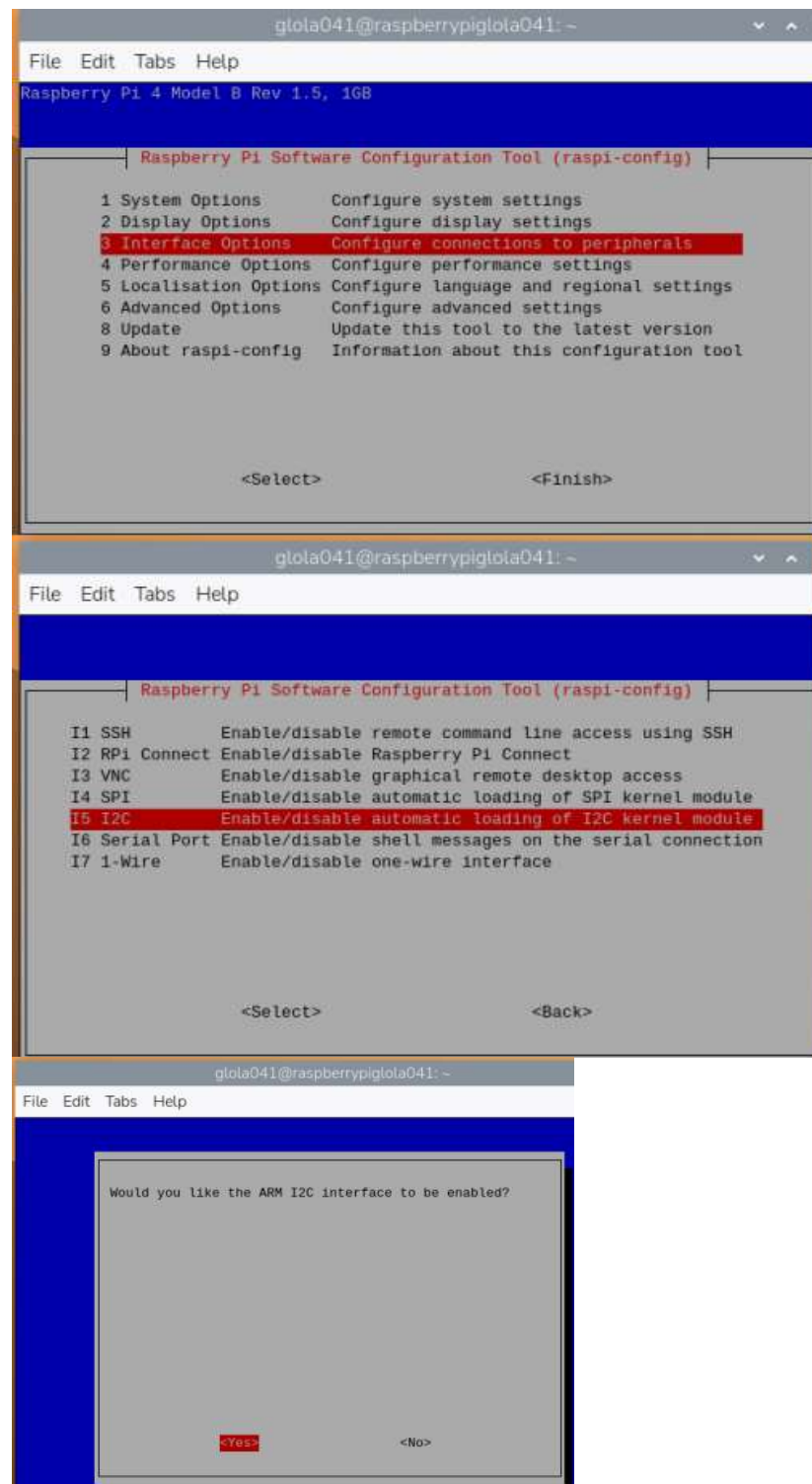

Enable the I²C interface

In the raspberry terminal enter:

→ `sudo raspi-config`

```
glola041@raspberrypi glola041:~ $ sudo raspi-config
```

Go to 3 interface Options and select I5 I2C



Docker installation

Download the script using the following command

→ `curl -fsSL https://get.docker.sh -o install-docker.sh`

Then verify the script content using:

→ `cat install-docker.sh`

```
glola041@raspberrypiglola041:~ $ curl -fsSL https://get.docker.com -o install-docker.sh
glola041@raspberrypiglola041:~ $ cat install-docker.sh
#!/bin/sh
set -e
# Docker Engine for Linux installation script.
```

Verify the script with :

→ `sh install-docker.sh --dry-run`

```
glola041@raspberrypiglola041:~ $ sh install-docker.sh --dry-run
```

Run the installation with:

→ `sudo sh install-docker.sh`

```
glola041@raspberrypiglola041:~ $ sudo sh install-docker.sh
```

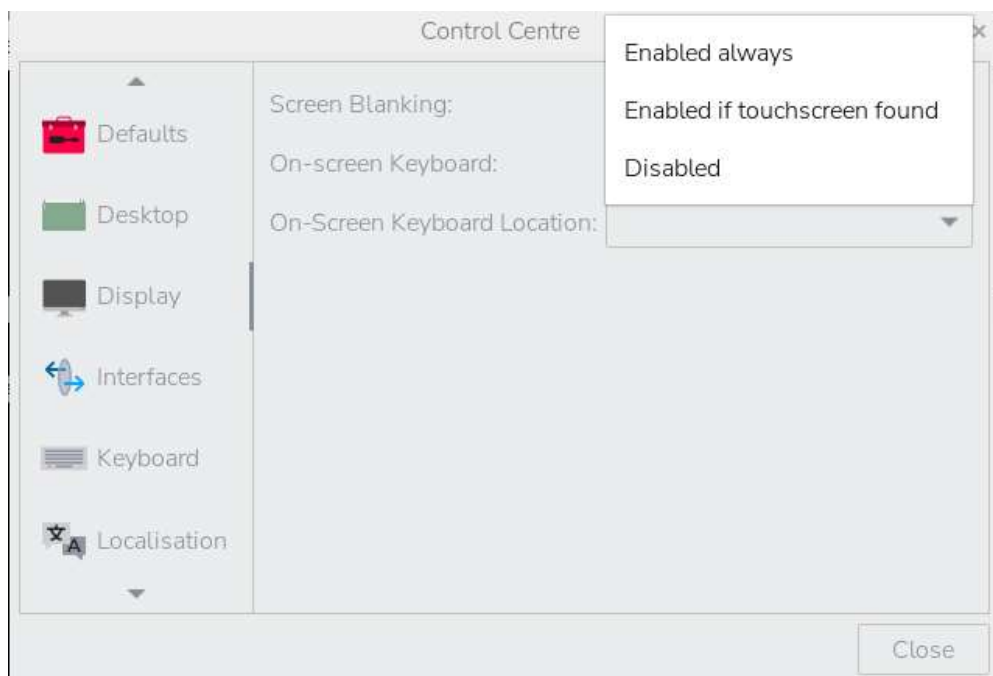
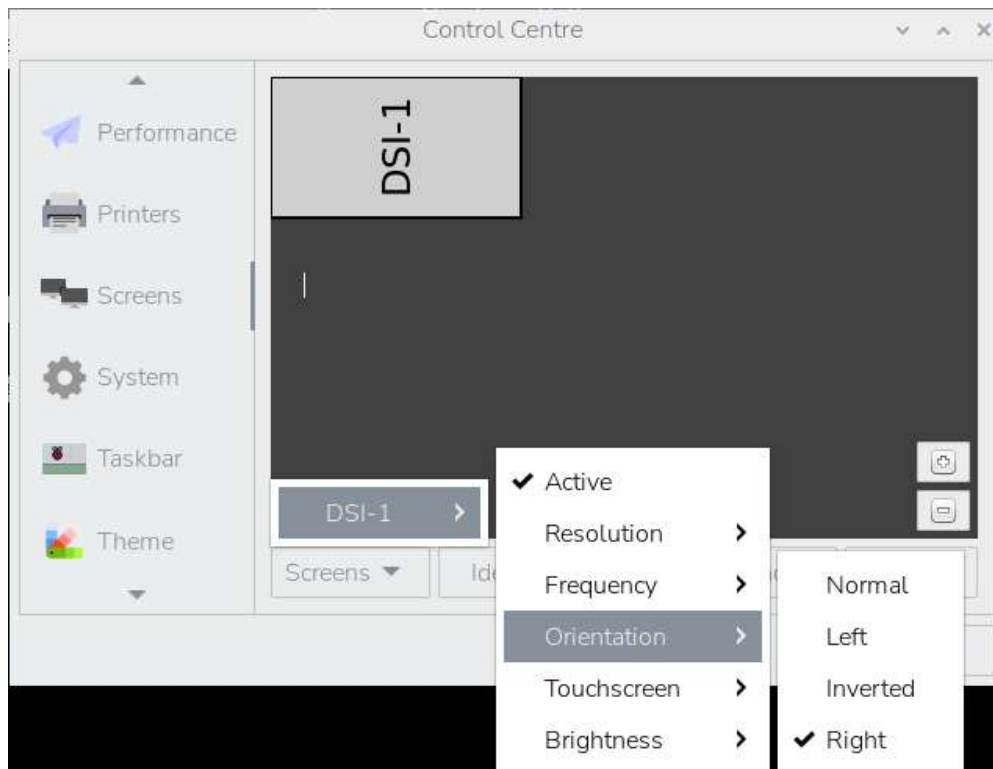
Verify if it's running with:

→ `sudo systemctl status docker`

```
glola041@raspberrypiglola041:~ $ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-11-19 08:50:36 CET; 1min 9s ago
     Invocation: 2133fc6198d14f15b6db1e8ce853821a
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 2631 (dockerd)
      Tasks: 10
         CPU: 760ms
    CGroup: /system.slice/docker.service
            └─2631 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd>
```

Screen orientation and keyboard

In control Center go to Screen and select the orientation to Right. To disable the keyboard go to Display and disable the on-screen keyboard.



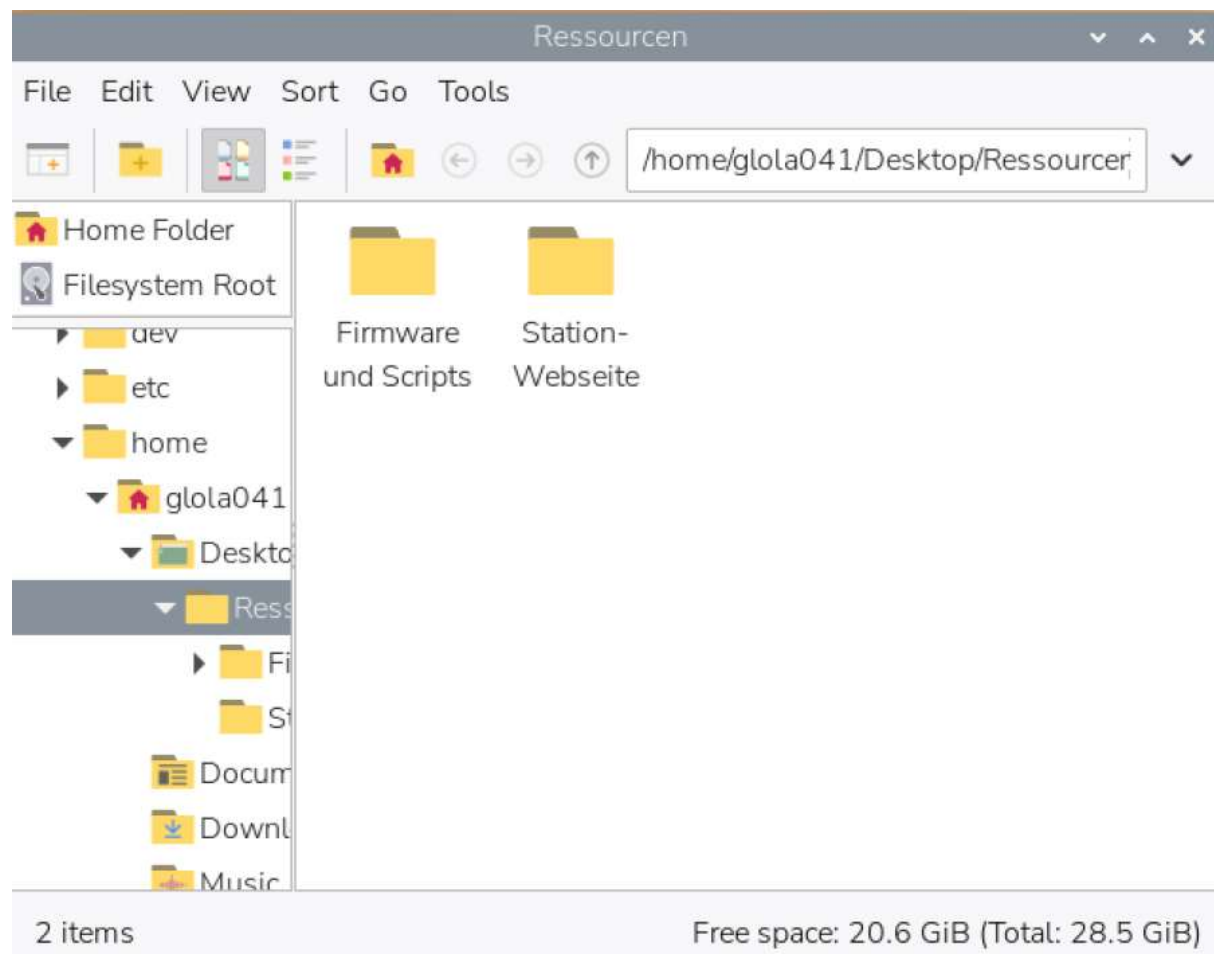
Copy the supplied files (firmware including configuration file, website, and scripts for testing the sensors) to the Raspberry Pi.

Create the folders /opt/station and /var/station. Ensure that the permissions are correct.

- ➔ `sudo mkdir /opt/station`
- ➔ `sudo mkdir /var/station`

```
glola041@raspberrypiglola041:~ $ sudo mkdir /opt/station
glola041@raspberrypiglola041:~ $ sudo mkdir /var/station
```

Download the the ressource file with as usb stick and move them in the desktop.



Move the 'firmware und script' in /opt/station and 'Station Webseite' in /opt/station/www. Enter those following commands:

```
➔ cd Desktop/  
ls  
cd Ressourcen  
sudo mv Firmware\ und\ Scripts\ /opt/station  
sudo mv Station-Website /opt/station/www  
cd Station-Website  
sudo mv index.php
```

```
glola041@raspberrypiglola041:~ $ cd Desktop/  
glola041@raspberrypiglola041:~/Desktop $ ls  
Ressourcen  
glola041@raspberrypiglola041:~/Desktop $ cd Ressourcen  
glola041@raspberrypiglola041:~/Desktop/Ressourcen $ sudo mv Firmware\ und\ Scripts/ /opt/station  
glola041@raspberrypiglola041:~/Desktop/Ressourcen $ ls  
Station-Webseite  
glola041@raspberrypiglola041:~/Desktop/Ressourcen $ sudo mv Station-Webseite/ /var/www/html  
glola041@raspberrypiglola041:~/Desktop/Ressourcen $
```

Note: I had to move the file, cause I placed it wrong directory, but the command given have the corrected version.

```
glola041@raspberrypiglola041:~ $ cd /var/www/html  
glola041@raspberrypiglola041:/var/www/html $  
glola041@raspberrypiglola041:/var/www/html $ sudo mv Station-Webseite/ /opt/station/www/
```

```
glola041@raspberrypiglola041:/opt/station $ cd 'Firmware und Scripts'/  
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ ls  
bh1750 bme280 config.toml measure.py mq135 test-bh1750.py test-bme280.py test-mq135.py  
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ sudo nano config.toml
```

```
glola041@raspberrypiglola041:/opt/station/www $ cd Station-Webseite/  
glola041@raspberrypiglola041:/opt/station/www/Station-Webseite $ ls  
index.php  
glola041@raspberrypiglola041:/opt/station/www/Station-Webseite $ sudo mv index.php ../  
glola041@raspberrypiglola041:/opt/station/www/Station-Webseite $ cd ../  
glola041@raspberrypiglola041:/opt/station/www $ ls  
index.php Station-Webseite  
glola041@raspberrypiglola041:/opt/station/www $
```

```
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts  
File Edit Tabs Help  
GNU nano 8.4 config.toml *  
#station_serial = # Serial number of the station
```

(Adjust the configuration file (/opt/station/config.toml) according to the work order (In particular, the serial number and I²C addresses will be configured later in the Atelier User Stories.) We were unable to do this part since there was no specification how to change the configuration of the serial number and I²C addresses.

How to create a docker container

To create the docker container you need to use this updated version from the teacher and Maksym Bylym:

➔ `docker run -d --name php --restart unless-stopped -p 8080:80 -v /tmp/station:/tmp/station -v /opt/station/www:/var/www/html php:8.4-apache`

```
glola041@raspberrypiglola041:~$ sudo docker run -d --name php --restart unless-stopped -p 8080:80 -v /tmp/station:/tmp/station -v /opt/station/www:/var/www/html php:8.4-apache
Unable to find image 'php:8.4-apache' locally
8.4-apache: Pulling from library/php
b89cf3ec7a3e: Pull complete
65dd1b4d69f9: Downloading [=====] 94.37MB/110.2MB
4f4fb700ef54: Download complete
22003554cbf4: Download complete
12ee6a10c3fa: Download complete
b6a214c39dd4: Pull complete
721a3cb0d966: Download complete
8caf2c340c63: Download complete
65dd1b4d69f9: Downloading [=====] 94.37MB/110.2MB
9f9: Downloading [=====] 95.42MB/110.2MB
65dd1b4d69f9: Downloading [=====] 96.47MB/110.2MB
65dd1b4d69f9: Downloading [=====] 97.52MB/110.2MB
65dd1b4d69f9: Extracting 2 s
65dd1b4d69f9: Extracting 3 s
65dd1b4d69f9: Extracting 4 s
65dd1b4d69f9: Pull complete
4f4fb700ef54: Pull complete
22003554cbf4: Pull complete
12ee6a10c3fa: Pull complete
c854e8ba3592: Download complete
721a3cb0d966: Pull complete
8caf2c340c63: Pull complete
def40a9103a7: Pull complete
d71c20bd3449: Pull complete
c854e8ba3592: Pull complete
7e0e8bc13627: Pull complete
53138ea265d8: Pull complete
```

To tell your Raspberry Pi what to present in kiosk mode and how to present it. Edit the file:

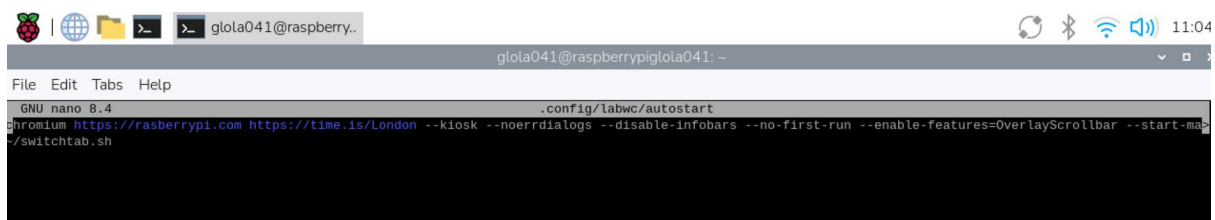
➔ `nano .config/labwc/autostart`

```
glola041@raspberrypiglola041:~$ sudo nano .config/labwc/autostart
```

To automatically run programs when the Raspberry Pi OS desktop has loaded.

Add the following two lines:

➔ `chromium http://localhost:8080 --kiosk --noerrdialogs --disable-infobars --no-first-run --enable-features=OverlayScrollbar --start-maximized &`
➔ `~/switchtab.sh`



The script will use the program we installed, wtype, to simulate and automate these keystrokes. Create the script with:

➔ `$ nano ~/switchtab.sh`

```
glola041@raspberrypiglola041:~$ sudo nano ~/switchtab.sh
```


And add the this in the file:

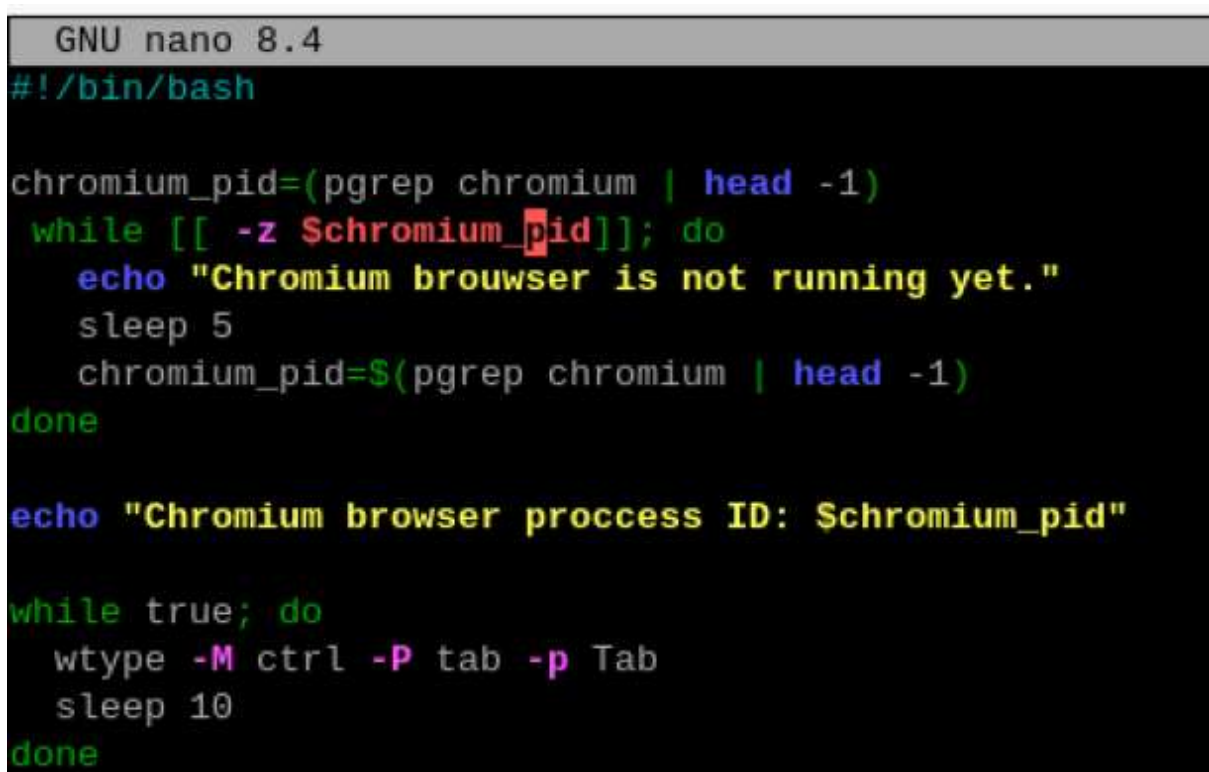
```
➔ #!/bin/bash

# Find Chromium browser process ID
chromium_pid=$(pgrep chromium | head -1)

# Check if Chromium is running
while [[ -z $chromium_pid ]]; do
    echo "Chromium browser is not running yet."
    sleep 5
    chromium_pid=$(pgrep chromium | head -1)
done

echo "Chromium browser process ID: $chromium_pid"

# Loop to send keyboard events
while true; do
    # Send Ctrl+Tab using `wtype` command
    wtype -M ctrl -P Tab -p Tab
    sleep 10
done
```



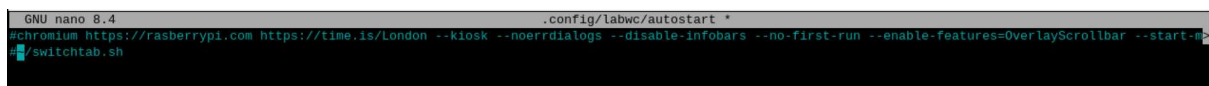
```
GNU nano 8.4
#!/bin/bash

chromium_pid=$(pgrep chromium | head -1)
while [[ -z $chromium_pid ]]; do
    echo "Chromium browser is not running yet."
    sleep 5
    chromium_pid=$(pgrep chromium | head -1)
done

echo "Chromium browser process ID: $chromium_pid"

while true; do
    wtype -M ctrl -P tab -p Tab
    sleep 10
done
```

Go back to the `.config/labwc/autostart` file and add the comment (#) to still be able to continue working on, else you will be redirected to a Kiosk webpage.



```
GNU nano 8.4 .config/labwc/autostart *
#chromium https://rasberrypi.com https://time.is/London --kiosk --noerrdialogs --disable-infobars --no-first-run --enable-features=OverlayScrollbar --start-page=switchtab.sh
```

Mark the `~/switchtab.sh` file as executable. Normally due to the fact that we added the comment to the `.config/labwc/autostart`, it should not now run. The reboot.

→ sudo chmod +x ~/switchtab.sh
sudo reboot

```
glola041@raspberrypiglola041:~ $ sudo chmod +x ~/switchtab.sh  
glola041@raspberrypiglola041:~ $ sudo reboot
```


Display the website in the kiosk mode

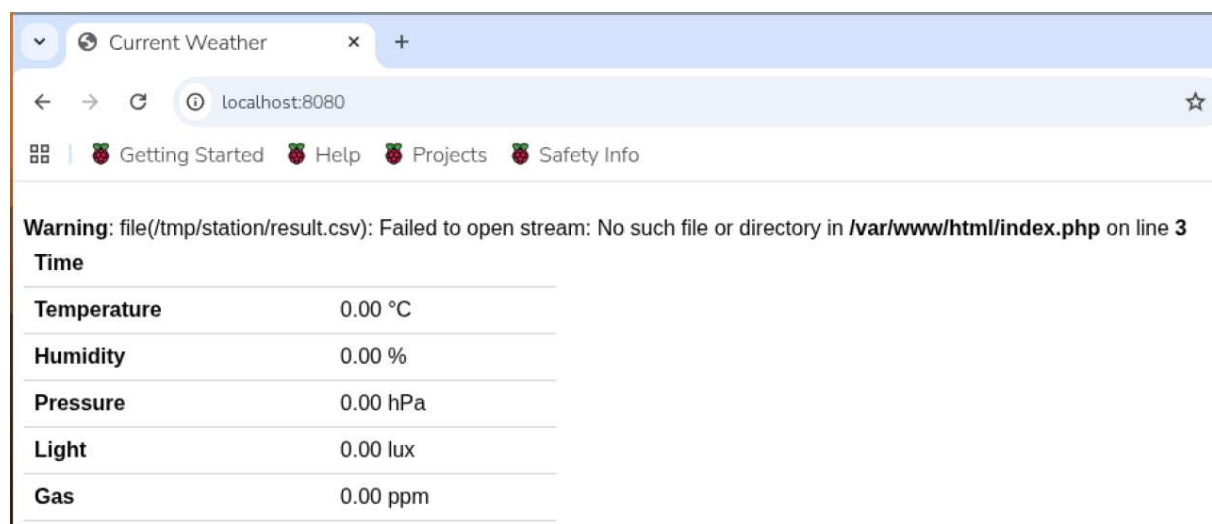
Go to cd tmp and create station by using:

```
→ sudo mkdir station
ls
cd station/
sudo nano result.csv
cd
cd /opt/station/
ls
cd 'Firmware und Scripts'/'
```

```
glola041@raspberrypiglola041:/tmp $ sudo mkdir station
glola041@raspberrypiglola041:/tmp $ ls
ssh-oMrjvQnpCIQA
station
systemd-private-6b75036022de4fab8e0c40d1c51438d3-bluetooth.service-GIUrVq
systemd-private-6b75036022de4fab8e0c40d1c51438d3-ModemManager.service-Vi3NtM
systemd-private-6b75036022de4fab8e0c40d1c51438d3-polkit.service-vCFHiP
systemd-private-6b75036022de4fab8e0c40d1c51438d3-systemd-logind.service-1EtMKC
wayvnc
glola041@raspberrypiglola041:/tmp $ cd station/
glola041@raspberrypiglola041:/tmp/station $ sudo nano result.csv
```

```
glola041@raspberrypiglola041:~ $ cd /opt/station/
glola041@raspberrypiglola041:/opt/station $ ls
'Firmware und Scripts'  www
glola041@raspberrypiglola041:/opt/station $ cd 'Firmware und Scripts'/'
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ ls
bh1750  config.toml  mq135          test-bme280.py
bme280  measure.py  test-bh1750.py test-mq135.py
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $
```

Now test it by opening it in your browser, entering localhost:8080



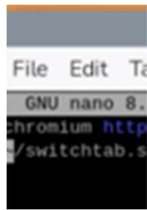
Warning: file(/tmp/station/result.csv): Failed to open stream: No such file or directory in /var/www/html/index.php on line 3

Time	
Temperature	0.00 °C
Humidity	0.00 %
Pressure	0.00 hPa
Light	0.00 lux
Gas	0.00 ppm

Once working get rid of the comment (#) in the front in your autostart file:

→ `sudo nano .config/labwc/autostart`

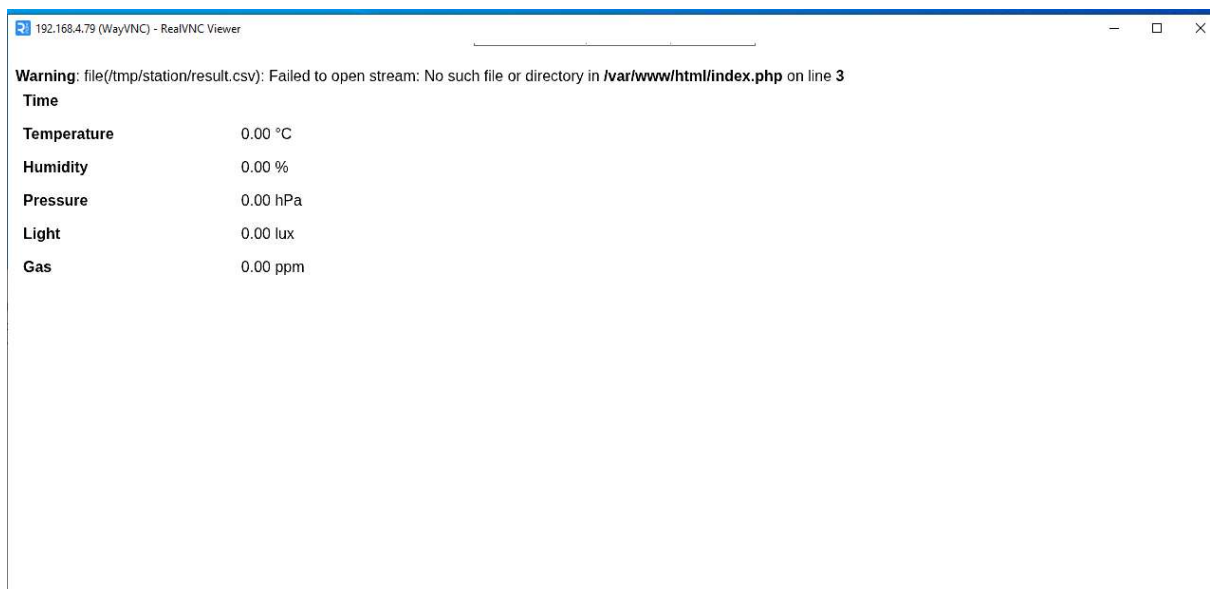
```
glo1a041@raspberrypiglola041:~$ sudo nano .config/labwc/autostart
```



Then enter:

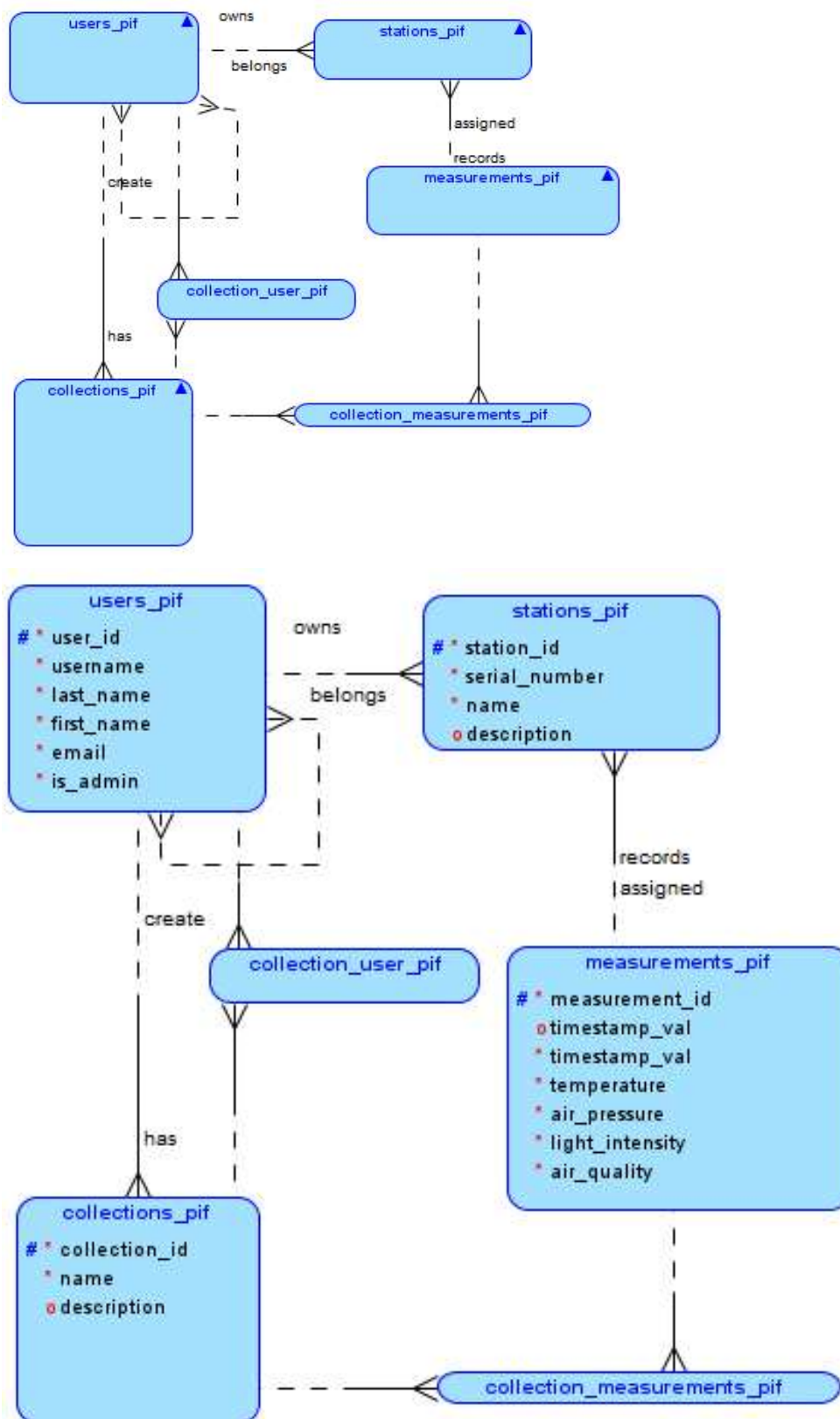
→ `Sudo reboot`

The website is now displayed in kiosk mode.

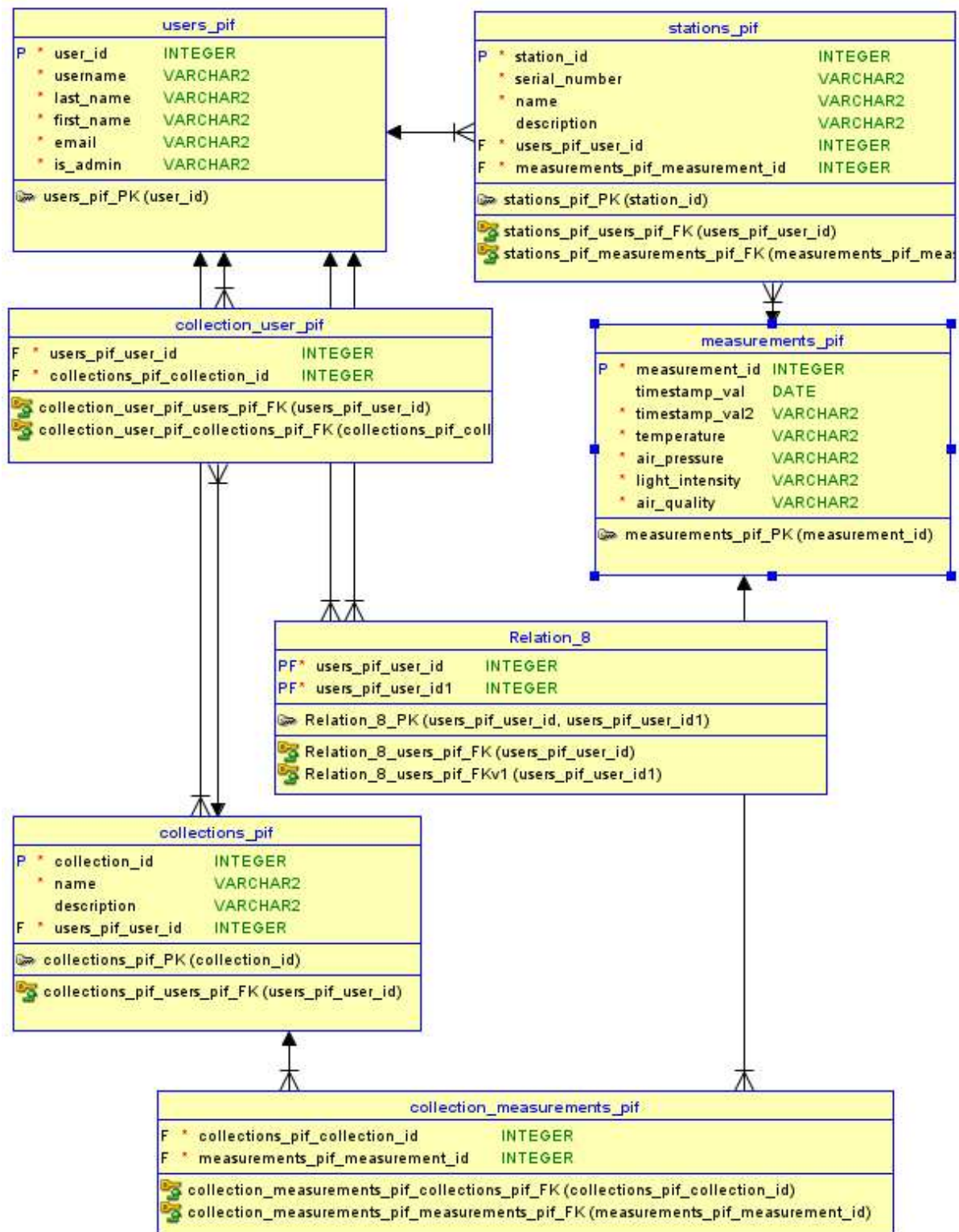


L2 – Database

Conceptual model (MCD/CDM)



Logical model (MLD / LDM)



Database – Apex syntax

```
CREATE TABLE users_pif (  
    user_id          VARCHAR2(50) PRIMARY KEY,  
    username         VARCHAR2(50) UNIQUE NOT NULL,  
    last_name        VARCHAR2(100) NOT NULL,  
    first_name       VARCHAR2(100) NOT NULL,  
    email            VARCHAR2(100) UNIQUE NOT NULL,  
    is_admin         CHAR(1)  
);
```

```
CREATE TABLE stations_pif (  
    station_id       NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    serial_number    VARCHAR2(20) UNIQUE NOT NULL,  
    name             VARCHAR2(100),  
    description      VARCHAR2(500),  
    user_id          VARCHAR2(50) NOT NULL,  
    CONSTRAINT fk_station_owneduser FOREIGN KEY (user_id) REFERENCES  
users_pif(user_id)  
);
```

```
CREATE TABLE measurements_pif (  
    measurement_id   NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    station_id       NUMBER NOT NULL,  
    timestamp_val    TIMESTAMP NOT NULL,  
    temperature      NUMBER(5,2),  
    humidity         NUMBER(5,2),  
    air_pressure     NUMBER(7,2),  
    light_intensity  NUMBER(10,2),  
    air_quality      NUMBER(10,2),  
    CONSTRAINT fk_measurements_station FOREIGN KEY (station_id) REFERENCES  
stations_pif(station_id)  
);
```

```
CREATE TABLE collections_pif (  
    collection_id    NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    name             VARCHAR2(100) NOT NULL,  
    description      VARCHAR2(500),  
    user_id          VARCHAR2(50) NOT NULL,  
    CONSTRAINT fk_collection_user FOREIGN KEY (user_id) REFERENCES  
users_pif(user_id)  
);
```

```
CREATE TABLE user_friends_pif (  
    user_id          VARCHAR2(50) NOT NULL,  
    friend_id        VARCHAR2(50) NOT NULL,  
    CONSTRAINT pk_user_friends PRIMARY KEY (user_id, friend_id),  
    CONSTRAINT fk_user_friends_user FOREIGN KEY (user_id) REFERENCES  
users_pif(user_id),  
    CONSTRAINT fk_user_friends_friend FOREIGN KEY (friend_id) REFERENCES  
users_pif(user_id)  
);
```

```
CREATE TABLE collection_user_pif (  
    collection_id    NUMBER NOT NULL,  
    user_id          VARCHAR2(50) NOT NULL,
```

```
CONSTRAINT pk_collection_user PRIMARY KEY (collection_id, user_id),
CONSTRAINT fk_cu_collection FOREIGN KEY (collection_id)
    REFERENCES collections_pif(collection_id),
CONSTRAINT fk_cu_user FOREIGN KEY (user_id)
    REFERENCES users_pif(user_id)
);
```

```
CREATE TABLE collection_measurements_pif (
    collection_id    NUMBER NOT NULL,
    measurement_id   NUMBER NOT NULL,
    CONSTRAINT pk_collection_measurements PRIMARY KEY (collection_id,
measurement_id),
    CONSTRAINT fk_cm_collection FOREIGN KEY (collection_id)
        REFERENCES collections_pif(collection_id),
    CONSTRAINT fk_cm_measurement FOREIGN KEY (measurement_id)
        REFERENCES measurements_pif(measurement_id)
);
```

Dataset – Apex syntax (IA creation)

Absolutely — here is a **complete, realistic sample dataset**. Everything below is **ready to copy-paste into Oracle SQL Developer or APEX**.

The data is realistic: users, stations, measurements, collections, shares, and friends.

✓ 1. USERS

```
INSERT INTO users_pif (user_id, username, last_name, first_name, email, is_admin)
VALUES ('u001', 'jdoe', 'Doe', 'John', 'john.doe@example.com', 'N');
```

```
INSERT INTO users_pif (user_id, username, last_name, first_name, email, is_admin)
VALUES ('u002', 'asmith', 'Smith', 'Alice', 'alice.smith@example.com', 'N');
```

```
INSERT INTO users_pif (user_id, username, last_name, first_name, email, is_admin)
VALUES ('u003', 'tadmin', 'Admin', 'Tom', 'tom.admin@example.com', 'Y');
```

```
INSERT INTO users_pif (user_id, username, last_name, first_name, email, is_admin)
VALUES ('u004', 'bwayne', 'Wayne', 'Bruce', 'bruce.wayne@example.com', 'N');
```

✓ 2. STATIONS

```
INSERT INTO stations_pif (serial_number, name, description, user_id)
VALUES ('SN-1001', 'Backyard Station', 'Outdoor weather station in the
backyard.', 'u001');
```

```
INSERT INTO stations_pif (serial_number, name, description, user_id)
VALUES ('SN-1002', 'Office Station', 'Station placed in the home office.',
'u001');
```

```
INSERT INTO stations_pif (serial_number, name, description, user_id)
VALUES ('SN-2001', 'Garden Station', 'Monitors temperature and humidity in the
garden.', 'u002');
```

```
INSERT INTO stations_pif (serial_number, name, description, user_id)
VALUES ('SN-3001', 'Batcave Climate System', 'Highly advanced monitoring
system.', 'u004');
```

✓ 3. MEASUREMENTS

```
INSERT INTO measurements_pif (station_id, timestamp_val, temperature, humidity,
air_pressure, light_intensity, air_quality)
VALUES (1, SYSTIMESTAMP-1, 22.5, 45.3, 1012.4, 350.0, 420);
```

```
INSERT INTO measurements_pif (station_id, timestamp_val, temperature, humidity,
air_pressure, light_intensity, air_quality)
VALUES (1, SYSTIMESTAMP-0.5, 23.1, 47.0, 1013.1, 500.0, 430);
```

```
INSERT INTO measurements_pif (station_id, timestamp_val, temperature, humidity,
air_pressure, light_intensity, air_quality)
VALUES (2, SYSTIMESTAMP-0.8, 21.9, 40.1, 1011.8, 120.0, 410);
```

```
INSERT INTO measurements_pif (station_id, timestamp_val, temperature, humidity,
air_pressure, light_intensity, air_quality)
VALUES (3, SYSTIMESTAMP-0.2, 18.3, 61.2, 1009.4, 900.0, 460);
```

```
INSERT INTO measurements_pif (station_id, timestamp_val, temperature, humidity,
air_pressure, light_intensity, air_quality)
VALUES (4, SYSTIMESTAMP-1.2, 16.0, 50.1, 1015.2, 5.0, 380);
```

✓ 4. COLLECTIONS

```
INSERT INTO collections_pif (name, description, user_id)
VALUES ('Summer Heat Readings', 'Collection of the hottest summer days.',
'u001');
```

```
INSERT INTO collections_pif (name, description, user_id)
VALUES ('Morning Measurements', 'Data collected every morning.', 'u002');
```

```
INSERT INTO collections_pif (name, description, user_id)
VALUES ('Batcave Experiments', 'Highly confidential climate data.', 'u004');
```

✓ 5. COLLECTION ↔ MEASUREMENT

```
INSERT INTO collection_measurements_pif VALUES (1, 1);
INSERT INTO collection_measurements_pif VALUES (1, 2);
INSERT INTO collection_measurements_pif VALUES (2, 3);
INSERT INTO collection_measurements_pif VALUES (2, 4);
INSERT INTO collection_measurements_pif VALUES (3, 5);
```

✓ 6. COLLECTION SHARING

```
INSERT INTO collection_user_pif VALUES (1, 'u002');
INSERT INTO collection_user_pif VALUES (2, 'u001');
```

✓ 7. FRIENDS LIST

```
INSERT INTO user_friends_pif VALUES ('u001', 'u002');
INSERT INTO user_friends_pif VALUES ('u002', 'u001');
INSERT INTO user_friends_pif VALUES ('u001', 'u004');
INSERT INTO user_friends_pif VALUES ('u004', 'u001');
INSERT INTO user_friends_pif VALUES ('u002', 'u003');
INSERT INTO user_friends_pif VALUES ('u003', 'u002');
```

Database – Phpmyadmin syntax

```
DROP DATABASE IF EXISTS PIF_Glola041;  
CREATE DATABASE PIF_Glola041;  
USE PIF_Glola041;
```

```
CREATE TABLE users_pif (  
    user_id      VARCHAR(50) PRIMARY KEY,  
    username     VARCHAR(50) UNIQUE NOT NULL,  
    last_name    VARCHAR(100) NOT NULL,  
    first_name   VARCHAR(100) NOT NULL,  
    email        VARCHAR(100) UNIQUE NOT NULL,  
    is_admin     CHAR(1)  
);
```

```
CREATE TABLE stations_pif (  
    station_id   INT AUTO_INCREMENT PRIMARY KEY,  
    serial_number VARCHAR(20) UNIQUE NOT NULL,  
    name         VARCHAR(100),  
    description  VARCHAR(500),  
    user_id      VARCHAR(50) NOT NULL,  
    CONSTRAINT fk_station_ownership  
        FOREIGN KEY (user_id) REFERENCES users_pif(user_id)  
);
```

```
CREATE TABLE measurements_pif (  
    measurement_id INT AUTO_INCREMENT PRIMARY KEY,  
    station_id     INT NOT NULL,  
    timestamp_val  DATETIME NOT NULL,  
    temperature    DECIMAL(5,2),  
    humidity       DECIMAL(5,2),  
    air_pressure   DECIMAL(7,2),  
    light_intensity DECIMAL(10,2),  
    air_quality    DECIMAL(10,2),  
    CONSTRAINT fk_measurements_station  
        FOREIGN KEY (station_id) REFERENCES stations_pif(station_id)  
);
```

```
CREATE TABLE collections_pif (  
    collection_id INT AUTO_INCREMENT PRIMARY KEY,  
    name          VARCHAR(100) NOT NULL,  
    description   VARCHAR(500),  
    user_id       VARCHAR(50) NOT NULL,  
    CONSTRAINT fk_collection_user  
        FOREIGN KEY (user_id) REFERENCES users_pif(user_id)  
);
```

```
CREATE TABLE user_friends_pif (  
    user_id      VARCHAR(50) NOT NULL,  
    friend_id    VARCHAR(50) NOT NULL,  
    PRIMARY KEY (user_id, friend_id),  
    CONSTRAINT fk_user_friends_user  
        FOREIGN KEY (user_id) REFERENCES users_pif(user_id),  
    CONSTRAINT fk_user_friends_friend  
        FOREIGN KEY (friend_id) REFERENCES users_pif(user_id)  
);
```

```
CREATE TABLE collection_user_pif (  
    collection_id INT NOT NULL,  
    user_id       VARCHAR(50) NOT NULL,  
    PRIMARY KEY (collection_id, user_id),
```

```
CONSTRAINT fk_cu_collection
    FOREIGN KEY (collection_id) REFERENCES collections_pif(collection_id),
CONSTRAINT fk_cu_user
    FOREIGN KEY (user_id) REFERENCES users_pif(user_id)
);

CREATE TABLE collection_measurements_pif (
    collection_id INT NOT NULL,
    measurement_id INT NOT NULL,
    PRIMARY KEY (collection_id, measurement_id),
    CONSTRAINT fk_cm_collection
        FOREIGN KEY (collection_id) REFERENCES collections_pif(collection_id),
    CONSTRAINT fk_cm_measurement
```

Dataset – Phpmyadmin syntax

Then enter the test sample generated by IA, the difference between apex is that it's timestamp_val in measurements_pif has to be modified to format work with phpmyadmin syntax

```
INSERT INTO users_pif (user_id, username, last_name, first_name, email, is_admin)
VALUES ('u001', 'jdoe', 'Doe', 'John', 'john.doe@example.com', 'N');
```

```
INSERT INTO users_pif (user_id, username, last_name, first_name, email, is_admin)
VALUES ('u002', 'asmith', 'Smith', 'Alice', 'alice.smith@example.com', 'N');
```

```
INSERT INTO users_pif (user_id, username, last_name, first_name, email, is_admin)
VALUES ('u003', 'tadmin', 'Admin', 'Tom', 'tom.admin@example.com', 'Y');
```

```
INSERT INTO users_pif (user_id, username, last_name, first_name, email, is_admin)
VALUES ('u004', 'bwayne', 'Wayne', 'Bruce', 'bruce.wayne@example.com', 'N');
```

```
INSERT INTO stations_pif (serial_number, name, description, user_id)
VALUES ('SN-1001', 'Backyard Station', 'Outdoor weather station in the
backyard.', 'u001');
```

```
INSERT INTO stations_pif (serial_number, name, description, user_id)
VALUES ('SN-1002', 'Office Station', 'Station placed in the home office.',
'u001');
```

```
INSERT INTO stations_pif (serial_number, name, description, user_id)
VALUES ('SN-2001', 'Garden Station', 'Monitors temperature and humidity in the
garden.', 'u002');
```

```
INSERT INTO stations_pif (serial_number, name, description, user_id)
VALUES ('SN-3001', 'Batcave Climate System', 'Highly advanced monitoring
system.', 'u004');
```

```
INSERT INTO measurements_pif
(station_id, timestamp_val, temperature, humidity, air_pressure, light_intensity,
air_quality)
VALUES
(1, NOW() - INTERVAL 1 DAY, 22.5, 45.3, 1012.4, 350.0, 420);
```

```
INSERT INTO measurements_pif
(station_id, timestamp_val, temperature, humidity, air_pressure, light_intensity,
air_quality)
VALUES
(1, NOW() - INTERVAL 12 HOUR, 23.1, 47.0, 1013.1, 500.0, 430);
```

```
INSERT INTO measurements_pif
(station_id, timestamp_val, temperature, humidity, air_pressure, light_intensity,
air_quality)
VALUES
(2, NOW() - INTERVAL 19 HOUR, 21.9, 40.1, 1011.8, 120.0, 410);
```

```
INSERT INTO measurements_pif
(station_id, timestamp_val, temperature, humidity, air_pressure, light_intensity,
air_quality)
VALUES
(3, NOW() - INTERVAL 5 HOUR, 18.3, 61.2, 1009.4, 900.0, 460);
```

```
INSERT INTO measurements_pif
```

```

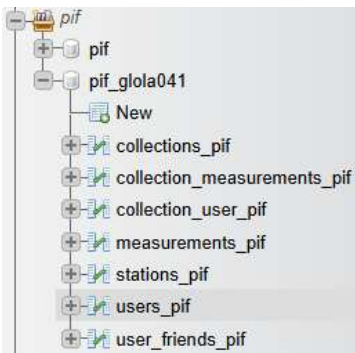
(station_id, timestamp_val, temperature, humidity, air_pressure, light_intensity,
air_quality)
VALUES
(4, NOW() - INTERVAL 1 DAY - INTERVAL 5 HOUR, 16.0, 50.1, 1015.2, 5.0, 380);
INSERT INTO collections_pif (name, description, user_id)
VALUES ('Summer Heat Readings', 'Collection of the hottest summer days.',
'u001');

INSERT INTO collections_pif (name, description, user_id)
VALUES ('Morning Measurements', 'Data collected every morning.', 'u002');

INSERT INTO collections_pif (name, description, user_id)
VALUES ('Batcave Experiments', 'Highly confidential climate data.', 'u004');
INSERT INTO collection_measurements_pif VALUES (1, 1);
INSERT INTO collection_measurements_pif VALUES (1, 2);
INSERT INTO collection_measurements_pif VALUES (2, 3);
INSERT INTO collection_measurements_pif VALUES (2, 4);
INSERT INTO collection_measurements_pif VALUES (3, 5);
INSERT INTO collection_user_pif VALUES (1, 'u002');
INSERT INTO collection_user_pif VALUES (2, 'u001');
INSERT INTO user_friends_pif VALUES ('u001', 'u002');
INSERT INTO user_friends_pif VALUES ('u002', 'u001');
INSERT INTO user_friends_pif VALUES ('u001', 'u004');
INSERT INTO user_friends_pif VALUES ('u004', 'u001');
INSERT INTO user_friends_pif VALUES ('u002', 'u003');
INSERT INTO user_friends_pif VALUES ('u003', 'u002');

```

Snapshot of the test sample in Phpmyadmin



There are all my tables.

✓ Showing rows 0 - 3 (4 total, Query took 0.0002 seconds.)

`SELECT * FROM `users_pif``

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

			user_id	username	last_name	first_name	email	is_admin	
☐	Edit	Copy	Delete	u001	jdoe	Doe	John	john.doe@example.com	N
☐	Edit	Copy	Delete	u002	asmith	Smith	Alice	alice.smith@example.com	N
☐	Edit	Copy	Delete	u003	tadmin	Admin	Tom	tom.admin@example.com	Y
☐	Edit	Copy	Delete	u004	bwayne	Wayne	Bruce	bruce.wayne@example.com	N

☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

✓ Showing rows 0 - 3 (4 total, Query took 0.0002 seconds.)

`SELECT * FROM `stations_pif``

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

			station_id	serial_number	name	description	user_id	
☐	Edit	Copy	Delete	1	SN-1001	Backyard Station	Outdoor weather station in the backyard.	u001
☐	Edit	Copy	Delete	2	SN-1002	Office Station	Station placed in the home office.	u001
☐	Edit	Copy	Delete	3	SN-2001	Garden Station	Monitors temperature and humidity in the garden.	u002
☐	Edit	Copy	Delete	4	SN-3001	Batcave Climate System	Highly advanced monitoring system.	u004

☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Showing rows 0 - 4 (5 total, Query took 0.0002 seconds.)

SELECT * FROM `measurements\_pif`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	measurement_id	station_id	timestamp_val	temperature	humidity	air_pressure	light_intensity	air_quality
☐ Edit Copy Delete	1	1	2025-12-03 10:19:07	22.50	45.30	1012.40	350.00	420.00
☐ Edit Copy Delete	2	1	2025-12-03 22:19:07	23.10	47.00	1013.10	500.00	430.00
☐ Edit Copy Delete	3	2	2025-12-03 15:19:07	21.90	40.10	1011.80	120.00	410.00
☐ Edit Copy Delete	4	3	2025-12-04 05:19:07	18.30	61.20	1009.40	900.00	460.00
☐ Edit Copy Delete	5	4	2025-12-03 05:19:07	16.00	50.10	1015.20	5.00	380.00

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Showing rows 0 - 2 (3 total, Query took 0.0003 seconds.)

SELECT * FROM `collections\_pif`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	collection_id	name	description	user_id
☐ Edit Copy Delete	1	Summer Heat Readings	Collection of the hottest summer days.	u001
☐ Edit Copy Delete	2	Morning Measurements	Data collected every morning.	u002
☐ Edit Copy Delete	3	Batcave Experiments	Highly confidential climate data.	u004

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Showing rows 0 - 4 (5 total, Query took 0.0002 seconds.)

SELECT * FROM `collection\_measurements\_pif`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	collection_id	measurement_id
☐ Edit Copy Delete	1	1
☐ Edit Copy Delete	1	2
☐ Edit Copy Delete	2	3
☐ Edit Copy Delete	2	4
☐ Edit Copy Delete	3	5

Check all | With selected: Edit Copy Delete Export

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

✓ Showing rows 0 - 1 (2 total, Query took 0.0002 seconds.)

```
SELECT * FROM `collection_user_pif`
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

		collection_id		user_id
☐	Edit	Copy	Delete	1 u002
☐	Edit	Copy	Delete	2 u001

↑ ☐ Check all With selected: Edit Copy

☐ Show all | Number of rows: 25 ▾ Filter rows:

✓ Showing rows 0 - 5 (6 total, Query took 0.0002 seconds.)

```
SELECT * FROM `user_friends_pif`
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▾ Filter rows: Search th

Extra options

		user_id		friend_id
☐	Edit	Copy	Delete	u001 u002
☐	Edit	Copy	Delete	u001 u004
☐	Edit	Copy	Delete	u002 u001
☐	Edit	Copy	Delete	u002 u003
☐	Edit	Copy	Delete	u003 u002
☐	Edit	Copy	Delete	u004 u001

↑ ☐ Check all With selected: Edit Copy D

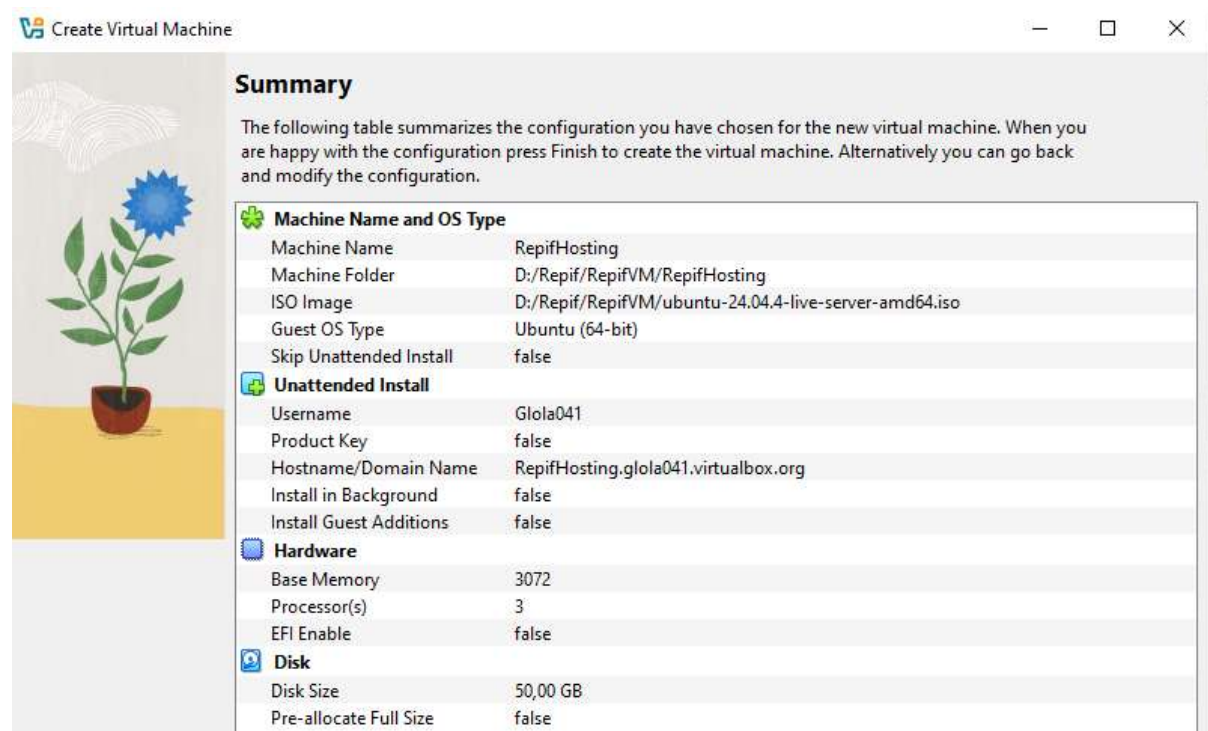
☐ Show all | Number of rows: 25 ▾ Filter rows: Search th

L4 - Web database server

Virtual or Physical Server?

I chose to use a virtual server that acts as a web and database server so that the Raspberry Pi does not need to be modified or overloaded with additional tasks. This keeps the sensor data collection and server functions separate.

Hosting the services on a virtual machine keeps the Raspberry Pi system clean, prevents performance issues, and allows changes without affecting the station directly.



```
Glola041@RepifHosting:~$ sudo timedatectl set-timezone Europe/Berlin
Glola041@RepifHosting:~$ timedatectl
          Local time: Thu 2026-02-26 11:30:13 CET
          Universal time: Thu 2026-02-26 10:30:13 UTC
             RTC time: Thu 2026-02-26 10:29:21
          Time zone: Europe/Berlin (CET, +0100)
System clock synchronized: yes
           NTP service: active
          RTC in local TZ: no
```


Installation steps

Apache:

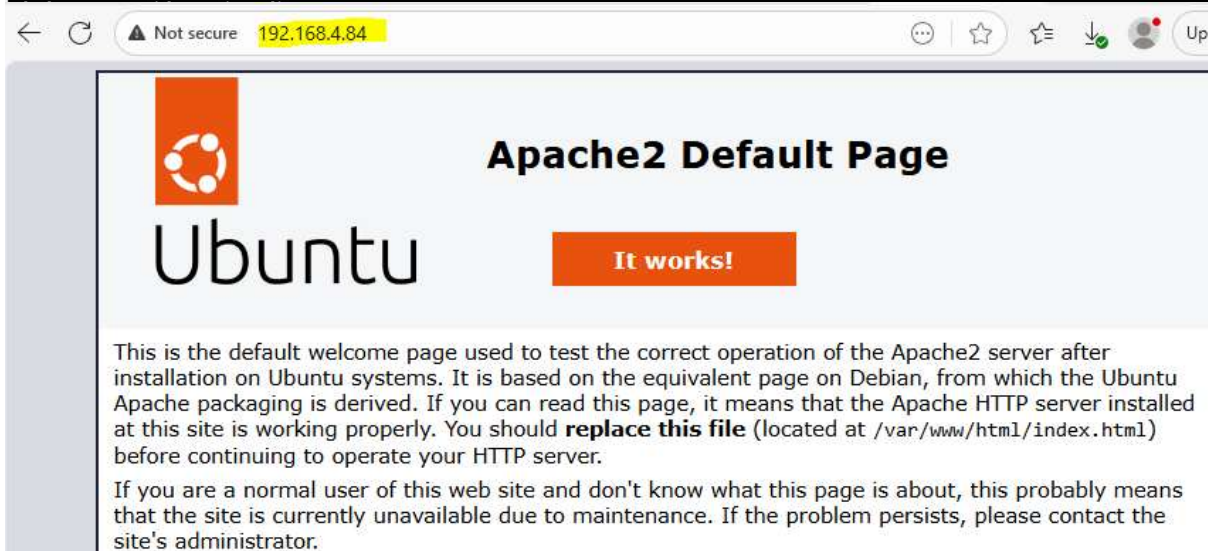
Deliver webpage through a browser. Verify the status.

- ➔ `sudo apt install apache2 -y`
- ➔ `apache2 -v`
- ➔ `sudo systemctl status apache2`

```
Glola041@RepifHosting:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-02-26 11:38:08 CET; 32s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 14670 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
  Main PID: 14673 (apache2)
    Tasks: 6 (limit: 3470)
   Memory: 13.8M (peak: 14.3M)
      CPU: 70ms
   CGroup: /system.slice/apache2.service
           └─14673 /usr/sbin/apache2 -k start
             └─14676 /usr/sbin/apache2 -k start
               └─14677 /usr/sbin/apache2 -k start
                 └─14678 /usr/sbin/apache2 -k start
                   └─14679 /usr/sbin/apache2 -k start
                     └─14680 /usr/sbin/apache2 -k start
```

Check for your ip adress with "ip a", and enter the address into a web browser. The apache2 Default page is displayed.

```
Glola041@RepifHosting:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:47:85:fb brd ff:ff:ff:ff:ff:ff
   inet 192.168.4.84/23 metric 100 brd 192.168.5.255 scope global dynamic enp0s3
       valid_lft 42521sec preferred_lft 42521sec
   inet6 fe80::a00:27ff:fe47:85fb/64 scope link
       valid_lft forever preferred_lft forever
```



MariaDB:

stores and manages the website's data. Verify the status.

➔ `sudo apt install mariadb-server -y`

```
Glola041@RepifHosting:~$ sudo systemctl status mariadb
• mariadb.service - MariaDB 10.11.14 database server
  Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
  Active: active (running) since Thu 2026-02-26 11:35:57 CET; 4min 38s ago
    Docs: man:mariadb(8)
          https://mariadb.com/kb/en/library/systemd/
  Main PID: 12217 (mariabdb)
  Status: "Taking your SQL requests now..."
```

PHP

Lets the server run interactive web pages. Install the extension to allow PHP to communicate with the MariaDB database.

➔ `sudo apt install php -y`

➔ `sudo apt install php-mysql -y`

phpMyAdmin:

For managing databases.

➔ `sudo apt install phpmyadmin -y`

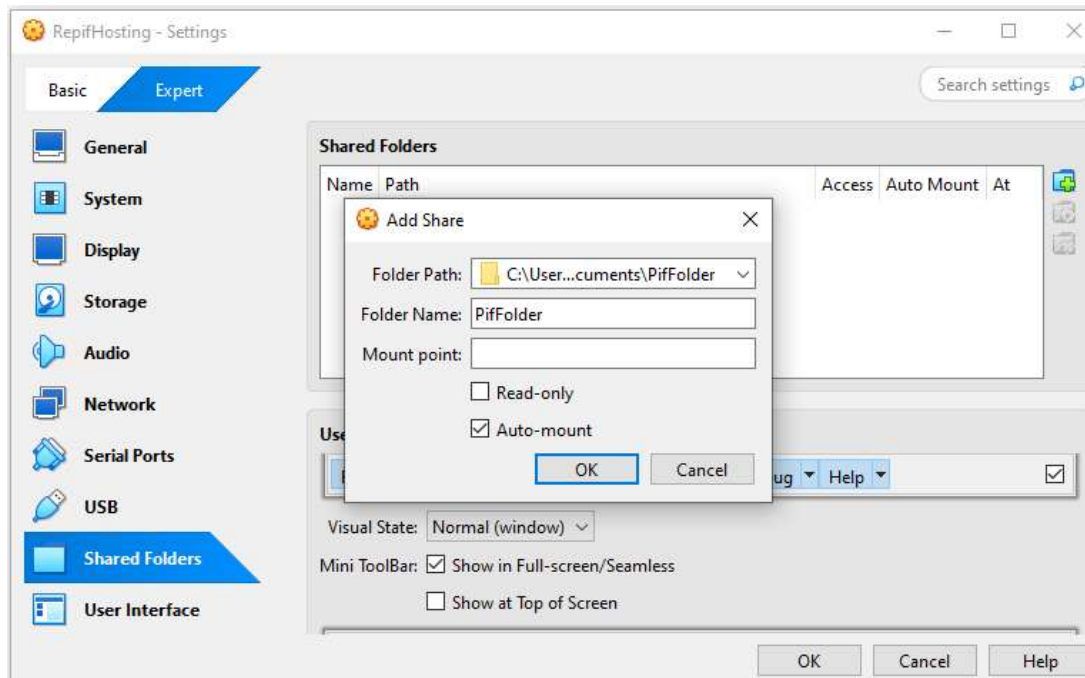
Install everything in one single command using:

➔ `sudo apt install apache2 mariadb-server php php-mysql phpmyadmin openssh-server -y`

```
Glola041@RepifHosting:~$ sudo apt install apache2 mariadb-server php php-mysql phpmyadmin openssh-server -y
```

Installation of a database and website – Share Folder Method

In Oracle VirtualBox Manager -> VM Settings -> Shared Folders, add a temporary shared folder that contains your website and database ZIP file. Put there your chosen path a folder you created earlier.



Install a program that extracts a .zip file using the following comand:

➔ `sudo apt install unzip -y`

```
Glola041@RepifHosting:~$ sudo apt install unzip -y
```

Shared folders are normaly mounted under /media and extract the ZIP file using the following command:

➔ `sudo unzip <YourZipName>.zip`

```
Glola041@RepifHosting:/media/Piffolder$ ls
L3-Website_Glola041.zip  test1.txt
Glola041@RepifHosting:/media/Piffolder$
Glola041@RepifHosting:/media/Piffolder$ sudo unzip L3-Website_Glola041.zip
```

Create the destination folder inside Apache and copy the extracted website in the new created path using the following command:

➔ `sudo mkdir -p /var/www/<YourChosenPathName>`

➔ `sudo cp -r <YourUnZipName>/* <YourChosenPathName>`

```
Glola041@RepifHosting:/media/Piffolder$ sudo mkdir -p /var/www/repifhosting
Glola041@RepifHosting:/media/Piffolder$ sudo cp -r Website_repif/* /var/www/repifhosting/
```

The rights to the website files should be set so that unauthorized persons cannot access them.

Give the owner Read, write, and execute permissions and Group and Others Read and execute permission:

- ➔ `sudo chown -R www-data:www-data /var/www/repifhosting/`
- ➔ `sudo chmod -R 755 /var/www/repifhosting/`

```
Glola041@RepifHosting:/media/Piffolder$ sudo chown -R www-data:www-data /var/www/repifhosting/
```

```
Glola041@RepifHosting:/var/www/repifhosting$ sudo chmod -R 755 /var/www/repifhosting
```

User and Database Creation in MariaDB

Insert your Database sql by opening MariaDB and importing the following command :

- ➔ `sudo mysql`
- ➔ `source <YourSqlFileName>.sql`

Check that the folder has correctly been imported with the following command:

- ➔ `SHOW DATABASES;`
- ➔ `SELECT * FROM <AnyTableName>;`

```
MariaDB [Project_Database]> source portableindoorfeedback.sql
MariaDB [Project_Database]> SHOW DATABASES;
+-----+
| Database |
+-----+
| Project_Database |
| information_schema |
| mysql |
| performance_schema |
| phpmyadmin |
| sys |
+-----+
6 rows in set (0.001 sec)

MariaDB [Project_Database]> USE Project_Database;
Database changed
MariaDB [Project_Database]> SELECT * FROM user;
Empty set (0.001 sec)

MariaDB [Project_Database]> SELECT * FROM userssdas;
ERROR 1146 (42S02): Table 'Project_Database.userssdas' doesn't exist
MariaDB [Project_Database]> SELECT * FROM users;
ERROR 1146 (42S02): Table 'Project_Database.users' doesn't exist
MariaDB [Project_Database]> SELECT * FROM collection;
Empty set (0.000 sec)
```


The website can be successfully accessed with a browser.

The Apache configuration file 000-default.conf needs to be changed so that the DocumentRoot deploys the folder /var/www/repifhosting, where the website files are stored. After saving the changes, the Apache service need to be restarted Check for your ip adress with "ip a", and enter the address into a web browser. The website is accessible but no data are displayed.

```
Glola041@RepifHosting:/etc/apache2/sites-enabled$ sudo nano 000-default.conf
```

```
<VirtualHost *:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/repifhosting

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf
</VirtualHost>
```

```
Glola041@RepifHosting:/etc/apache2/sites-enabled$ sudo systemctl restart apache2
```

```
Glola041@RepifHosting:/etc/apache2/sites-enabled$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:47:85:fb brd ff:ff:ff:ff:ff:ff
    inet 192.168.4.84/23 metric 100 brd 192.168.5.255 scope global dynamic enp0s3
        valid_lft 41111sec preferred_lft 41111sec
    inet6 fe80::a00:27ff:fe47:85fb/64 scope link
        valid_lft forever preferred_lft forever
Glola041@RepifHosting:/etc/apache2/sites-enabled$
```


Index of /



	<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
	Admin.php	2026-02-26 15:57	1.4K	
	AdminCollections.php	2026-02-26 15:57	3.8K	
	AdminCollectionsAddMeasurements.php	2026-02-26 15:57	5.0K	
	Collection.php	2026-02-26 15:57	1.4K	
	CommonCode.php	2026-03-11 08:44	20K	
	FriendPage.php	2026-02-26 15:57	11K	
	Index.php	2026-02-26 15:57	5.9K	
	ManageStationAdmin.php	2026-02-26 15:57	5.3K	
	Menu.php	2026-02-26 15:57	1.3K	
	MyCollections.php	2026-02-26 15:57	10K	
	MyStation.php	2026-02-26 15:57	5.1K	
	SharedCollections.php	2026-02-26 15:57	2.1K	
	StationAdmin.php	2026-02-26 15:57	2.6K	
	UserManagement.php	2026-02-26 15:57	6.6K	
	banner.jpg	2026-02-26 15:57	26K	
	collection_view.php	2026-02-26 15:57	3.6K	
	collections_create.php	2026-02-26 15:57	5.0K	
	measurements.php	2026-02-26 15:57	6.0K	
	portableindoorfeedback.sql	2026-02-26 15:57	36K	
	register.php	2026-02-26 15:57	3.1K	
	style.css	2026-02-26 15:57	1.8K	

The database username and password needs to be changed to the newly created database user so that the website does not use the administrator (root) account. Now after restarting Apache the website are displaying the data.

GNU nano 7.2CommonCode.php *

```
<?php
// Start session for auth and preferences
session_start();
if (!isset($_SESSION["UserLoggedIn"])) {
    $_SESSION["UserLoggedIn"] = false;
}

// Default language selection
if (!isset($_SESSION["language"])) {
    $_SESSION["language"] = "EN"; //english language by default
}

if (isset($_GET["language"])) {
    $_SESSION["language"] = $_GET["language"];
}

// Default role if not set in session
if (!isset($_SESSION["role"])) {
    $_SESSION["role"] = "User"; // safe default (not Admin)
}

// https://www.w3schools.com/php/php_sessions.asp? (PHP Session Functions)
// https://www.w3schools.com/php/php_mysql_prepared_statements.asp? (PHP MySQL Prepared Statements)

// Database connection settings
$host = "localhost";
$username = "UserTest";
$password = "UserTest123!";
$dbName = "Project_Database";
```

Not secure 192.168.4.84/Index.php

Portable Indoor Feedback



Please enter your username and password here :

Login

Don't have an account? [Register here](#)

MySQL should be secured with mysql_secure_installation

The mysql_secure_installation was used to remove insecure default settings and to improve the security for the database server. Run the command:

➔ sudo mysql_secure_installation

```
Glo1a041@RepifHosting:/var/www/repifhosting$ sudo mysql_secure_installation
```

Use the following recommended answers:

- ➔ Enter current password → press enter
- ➔ Switch to unix_socket authentication → Yes
- ➔ Change root password → No
- ➔ Remove anonymous users → Yes
- ➔ Disallow root login remotely → Yes
- ➔ Remove test database → Yes
- ➔ Reload privilege tables → Yes

```
Glola041@RepifHosting:/var/www/repifhosting$ sudo mysql_secure_installation
```

```
NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB  
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!
```

In order to log into MariaDB to secure it, we'll need the current password for the root user. If you've just installed MariaDB, and haven't set the root password yet, you should just press enter here.

```
Enter current password for root (enter for none):  
OK, successfully used password, moving on...
```

Setting the root password or using the unix_socket ensures that nobody can log into the MariaDB root user without the proper authorisation.

You already have your root account protected, so you can safely answer 'n'.

```
Switch to unix_socket authentication [Y/n] y  
Enabled successfully!  
Reloading privilege tables..  
... Success!
```

You already have your root account protected, so you can safely answer 'n'.

```
Change the root password? [Y/n] n  
... skipping.
```

By default, a MariaDB installation has an anonymous user, allowing anyone to log into MariaDB without having to have a user account created for them. This is intended only for testing, and to make the installation go a bit smoother. You should remove them before moving into a production environment.

```
Remove anonymous users? [Y/n] y  
... Success!
```

Normally, root should only be allowed to connect from 'localhost'. This ensures that someone cannot guess at the root password from the network.

```
Disallow root login remotely? [Y/n] y  
... Success!
```

By default, MariaDB comes with a database named 'test' that anyone can access. This is also intended only for testing, and should be removed before moving into a production environment.

```
Remove test database and access to it? [Y/n] y  
- Dropping test database...  
... Success!  
- Removing privileges on test database...  
... Success!
```

Reloading the privilege tables will ensure that all changes made so far will take effect immediately.

```
Reload privilege tables now? [Y/n] y
```

All installed packages and the server's operating system should be up to date

```
Glola041@RepifHosting:/var/www/repifhosting$ sudo apt update
Hit:1 https://lu.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 https://lu.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 https://lu.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Glola041@RepifHosting:/var/www/repifhosting$ sudo apt upgrade -y
```

User Story

Must

- One virtual or physical server is running Apache, MariaDB, and PHP.
- The website and database are installed and working.
- The website is accessible through a web browser.
- A test database user is created for the website.
- Basic file permissions are configured to protect the website files.

Should

- The database server is secured with `mysql_secure_installation`.
- All installed packages and the server's operating system are up to date.
- phpMyAdmin is manually installed

Could

- A self-signed certificate can be created and the virtual host can be configured to use HTTPS.
- HTTP is redirected to HTTPS.

L5 – Data Transfer

PHP Script for Data Transfer

The explanation of how the receive_data.php script works is written in the snapshot as a comments in the code.

```
<?php
// makes the connection to the global database connection
require_once 'CommonCode.php';
// set the response to JSON format
header('Content-Type: application/json');

// Get the measurement data sent via HTTP POST from the Raspberry Pi
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    exit("Only POST requests are to be accepted.");
}
$station_serial = $_POST['station_serial'];
$timestamp      = $_POST['timestamp'];
$temperature    = $_POST['temperature'];
$humidity       = $_POST['humidity'];
$pressure       = $_POST['pressure'];
$light          = $_POST['light'];
$gas            = $_POST['gas'];

// Insert the received data into the measurement table
$sql = "INSERT INTO measurement (temperature, humidity, pressure, light, gas, timestamp, fk_station_records)
VALUES (?, ?, ?, ?, ?, ?, ?)";

// Prepare and execute the SQL statement
$stmt = $connection->prepare($sql);
$stmt->bind_param("dddddss", $temperature, $humidity, $pressure, $light, $gas, $timestamp, $station_serial);
$stmt->execute();

// Send a success response back to the Raspberry Pi
echo json_encode([
    "success" => true,
    "message" => "Measurement saved successfully."
]);
?>
```

Manual Data Entry Form for Measurement

A sender.html page was created with default sample data inside it to avoid manually having to rewrite the same test data each time. This page allows us to imitate a station sending measurements using an HTTP POST request.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Measurement Sender Test</title>
</head>
<body>

<h2>Send Measurement Data (Test)</h2>

<form method="POST" action="receiver.php">

  Serial Number:<br>
  <input type="text" name="station_serial" value="STATION001"><br><br>

  Timestamp (YYYY-MM-DD HH:MM:SS):<br>
  <input type="text" name="timestamp" value="2025-12-08 10:30:00"><br><br>

  Temperature:<br>
  <input type="text" name="temperature" value="22.5"><br><br>

  Humidity:<br>
  <input type="text" name="humidity" value="45.2"><br><br>

  Pressure:<br>
  <input type="text" name="pressure" value="1013.8"><br><br>

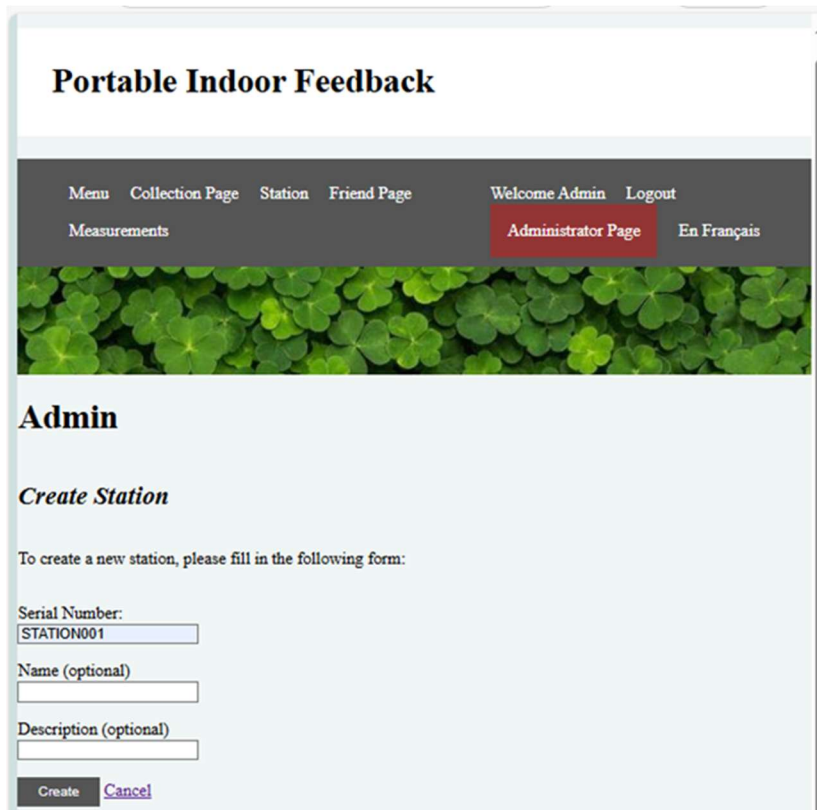
  Light:<br>
  <input type="text" name="light" value="350"><br><br>

  Gas:<br>
  <input type="text" name="gas" value="0.75"><br><br>

  <input type="submit" value="Send Data">

</form>
</body>
</html>
```


Before sending the sample data the station needs to be added by an administrator. Allowing one user to claim the station and access the received test data.



Portable Indoor Feedback

Menu Collection Page Station Friend Page Welcome Admin Logout
Measurements Administrator Page En Français

Admin

Create Station

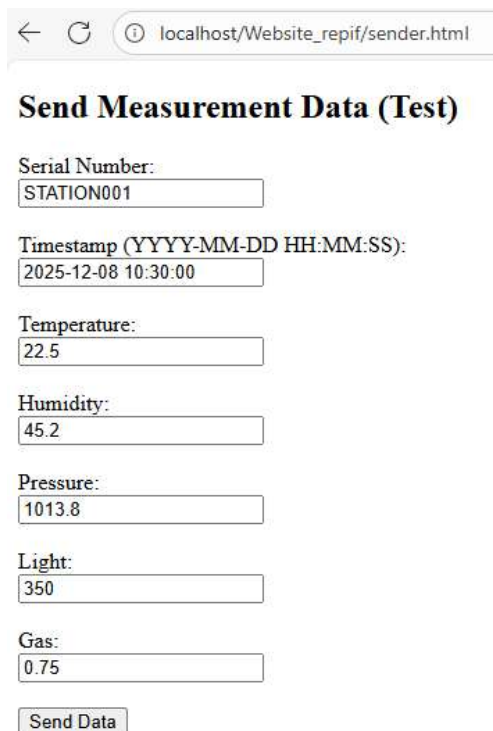
To create a new station, please fill in the following form:

Serial Number:

Name (optional)

Description (optional)

Create Cancel



Send Measurement Data (Test)

Serial Number:

Timestamp (YYYY-MM-DD HH:MM:SS):

Temperature:

Humidity:

Pressure:

Light:

Gas:

Send Data



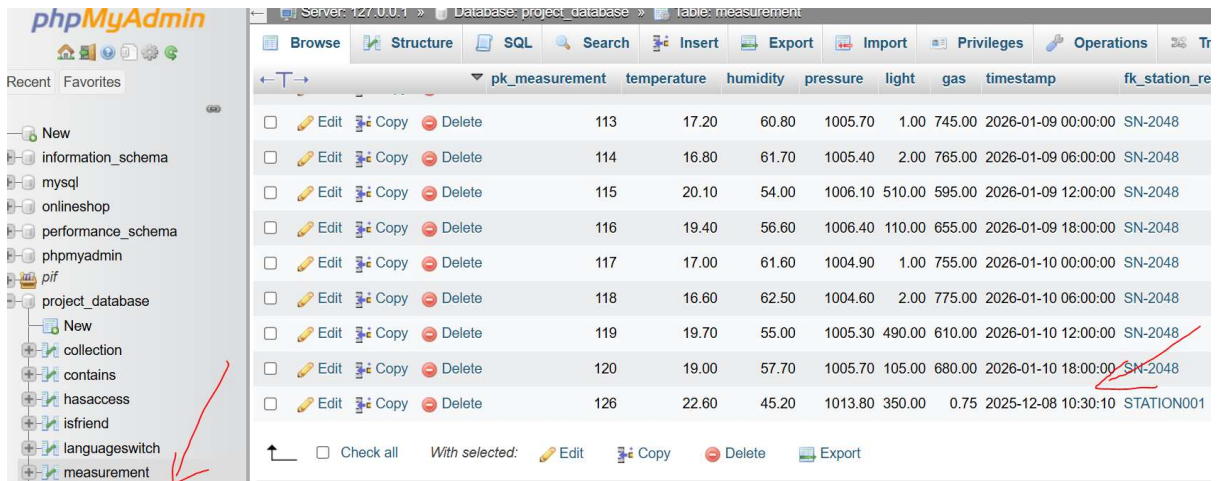
localhost/Website_repif/receiver.php

Pretty-print ☐

```
{"success":true,"message":"Measurement saved successfully."}
```

The sample test data were successfully transmitted and stored in the database inside the **measurement** table. The success message indicates that the data insertion has been completed.

The test sample data is transferred correctly and stored in the database in the table messurment.



phpMyAdmin

Server: 127.0.0.1 » Database: project_database » Table: measurement

Recent Favorites

New

- information_schema
- mysql
- onlineshop
- performance_schema
- phpmyadmin
- piif
- project_database
 - New
 - collection
 - contains
 - hasaccess
 - isfriend
 - languageswitch
 - measurement

	pk_measurement	temperature	humidity	pressure	light	gas	timestamp	fk_station_re
☐	Edit Copy Delete	113	17.20	60.80	1005.70	1.00	745.00	2026-01-09 00:00:00 SN-2048
☐	Edit Copy Delete	114	16.80	61.70	1005.40	2.00	765.00	2026-01-09 06:00:00 SN-2048
☐	Edit Copy Delete	115	20.10	54.00	1006.10	510.00	595.00	2026-01-09 12:00:00 SN-2048
☐	Edit Copy Delete	116	19.40	56.60	1006.40	110.00	655.00	2026-01-09 18:00:00 SN-2048
☐	Edit Copy Delete	117	17.00	61.60	1004.90	1.00	755.00	2026-01-10 00:00:00 SN-2048
☐	Edit Copy Delete	118	16.60	62.50	1004.60	2.00	775.00	2026-01-10 06:00:00 SN-2048
☐	Edit Copy Delete	119	19.70	55.00	1005.30	490.00	610.00	2026-01-10 12:00:00 SN-2048
☐	Edit Copy Delete	120	19.00	57.70	1005.70	105.00	680.00	2026-01-10 18:00:00 SN-2048
☐	Edit Copy Delete	126	22.60	45.20	1013.80	350.00	0.75	2025-12-08 10:30:10 STATION001

Check all With selected: Edit Copy Delete Export

Installation of the PHP Data Transfer Receiver Script on the Server

```
Glola041@RepifHosting:/media/Piffolder$ ls
L3-Website_Glola041.zip  recieve_data.php  recieve.zip  test1.txt  Website_repif
Glola041@RepifHosting:/media/Piffolder$ sudo cp -r recieve_data.php /var/www/repifhosting/

Glola041@RepifHosting:/var/www/repifhosting$ sudo mv recieve_data.php receive_data.php
Glola041@RepifHosting:/var/www/repifhosting$ ls -l receive_data.php
-rwxr-x--- 1 root root 1203 Mar 18 09:27 receive_data.php
Glola041@RepifHosting:/var/www/repifhosting$ sudo chmod 644 /var/www/repifhosting/receive_data.php
Glola041@RepifHosting:/var/www/repifhosting$ sudo chown www-data:www-data /var/www/repifhosting/receive_data.php
```

Using the shared folder installed in the previous sprint, I unzip the PHP script for data transfer called `receive_data.php` and place it in my `repifHosting` folder. I then assign the correct permissions, similar to the other files configured in the previous sprint. This is done using the following commands:

- ➔ `sudo cp -r receive_data.php /var/www/repifHosting/`
- ➔ `sudo chmod 644 /var/www/repifHosting/receive_data.php`
- ➔ `sudo chown www-data:www-data /var/www/repifHosting/receive_data.php`

Station Configuration for Data Transfer

```
glola041@raspberrypiglola041: /opt/station/Firmware und Scripts
GNU nano 8.4 config.toml
station_serial = "STATION001" # Serial number of the station

interval = 2 # Time between measurements

toggle_intensity = 100 # The light level at which the LED should be turned on

[i2c] # I2C-Addresses of the sensors
bmp280 = 0x76
bh1750 = 0x23
mq135 = 0x48

[destination] # Information about the server
ip = "192.168.4.91" # IP-Address or Hostname of the Server
path = "/receive_data.php" # Path to the script that accepts the measurements
```

On the Raspberry Pi station, the configuration file (**config.toml**) needs to be updated with the IP address of the virtual machine and the path to the PHP receiver script in quotes.

The path must corresponds to the location of the receive_data.php in the website.

Station: STATION001 - v

Date Range: Today v

Apply

Station	Timestamp	Temperature	Humidity	Pressure	Light	Gas	Delete
STATION001	2026-04-15 08:57:00	20.00 °C	50.00 %	1000.00 hPa	0.00 lux	500.00 ppm	Delete
STATION001	2026-04-15 08:56:58	20.00 °C	50.00 %	1000.00 hPa	0.00 lux	500.00 ppm	Delete
STATION001	2026-04-15 08:56:55	20.00 °C	50.00 %	1000.00 hPa	0.00 lux	500.00 ppm	Delete
STATION001	2026-04-15 08:56:53	20.00 °C	50.00 %	1000.00 hPa	0.00 lux	500.00 ppm	Delete
STATION001	2026-04-15 08:56:51	20.00 °C	50.00 %	1000.00 hPa	0.00 lux	500.00 ppm	Delete
STATION001	2026-04-15 08:56:49	20.00 °C	50.00 %	1000.00 hPa	0.00 lux	500.00 ppm	Delete
STATION001	2026-04-15 08:56:47	20.00 °C	50.00 %	1000.00 hPa	0.00 lux	500.00 ppm	Delete

The station data is now accessible on webserver virtual machine.

Errors

During testing, the station was able to send data to another student's server when using their IP address and path. I was not able to transfer the data to my own server, even though I could access the receiver script. The station configuration was correct. After further investigation, it was found that the issue was related to the firewall settings of my PC, which were blocking incoming connections from other devices. This explains why I was able to access other servers, but other devices, including the station, were not able to properly communicate with mine.

L6 - Backup Server

Objective

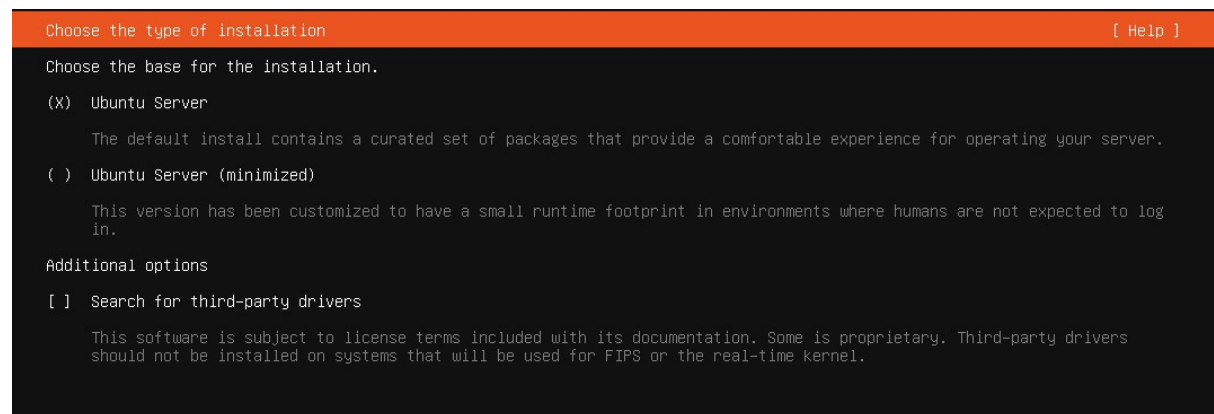
For this user story we need to configure a backup server able to store daily backups of the project database. The backup had to be created every day at 8:00 a.m. and stored on the backup server.

The data transfer path used is

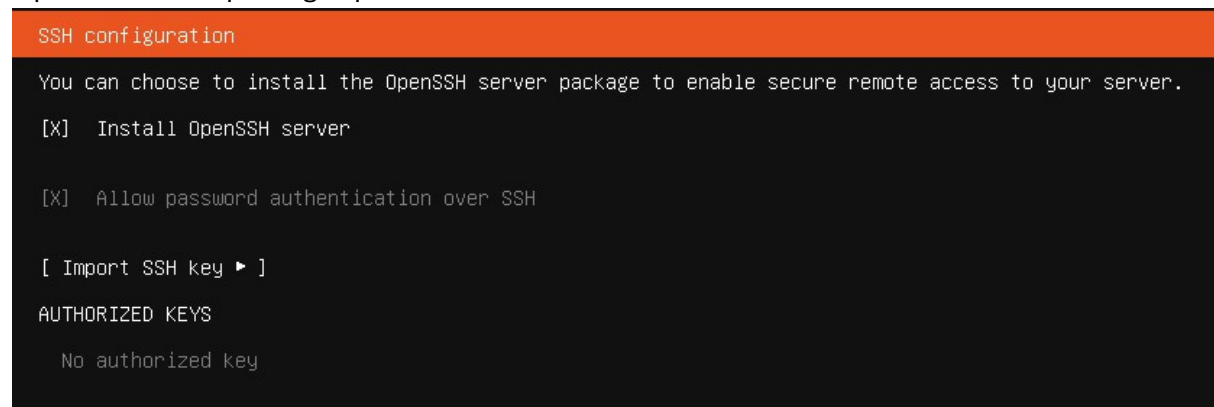
Backup server → connects to database server → creates and stores backup locally

Backup server installation

A new virtual machine needs to be created for the backup server. The current LTS version of Ubuntu Server and the OpenSSH server have been installed together.



OpenSSH server package option:



Installing MySQL client and FTP software.

The installation of the MySQL client is required for the backup to connect to the web/database server to later run mysqldump remotely.

Mysqldump is a command-line tool to create backups of MySQL databases. This tool is commonly used for backups, migrations, and transferring databases between servers. (link: [MySQL :: MySQL 8.4 Reference Manual :: 6.5.4 mysqldump — A Database Backup Program](#))

➔ sudo apt install mysql-client

```
glola041@glola041backup:~$ sudo apt install mysql-client
```

FTP software was also later installed to provide access to the backup directory.

➔ sudo apt install vsftpd

Creating a MySQL backup user on the web/database server

On the web/database server, create a MySQL user. The user was configured so it is only allowed to access the MySQL database from the backup server. Connecting the backup server and access the project database (The name of my database). Separating the backup process to the rest of the main database. The user is restricted to the 192.168.% network, to correspond to the local network used by the virtual machines, in case the local network changes.

➔ CREATE USER 'BackupUser'@'192.168.%' IDENTIFIED BY 'BackEme1';

➔ GRANT ALL PRIVILEGES ON Project_Database.* TO 'BackupUser'@'192.168.%;'

➔ FLUSH PRIVILEGES;

```
MariaDB [(none)]> CREATE USER 'BackupUser'@'192.168.%' IDENTIFIED BY 'BackEme1';
Query OK, 0 rows affected (0.006 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON Project_Database.* TO 'BackupUser'@'192.168.%;'
Query OK, 0 rows affected (0.004 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.001 sec)
```

Creating a backup directory

Create a Backup directory on the backup server to store the SQL backup files

➔ sudo mkdir -p /var/backups/project_db

➔ sudo chown \$USER:\$USER /var/backups/project_db

```
glola041@glola041backup:~$ sudo mkdir -p /var/backups/project_db
glola041@glola041backup:~$ sudo chown $USER:$USER /var/backups/project_db
```


Creating and testing the backup script

Create a script called `backup_db.sh`. Its role will be to connect to the database server, create a dump of the project database using `mysqldump`. This will pull the data from the database and save the result in the backup directory with a date and time in the filename.

```
#!/bin/bash
```

```
DATE=$(date +%F_%H-%M-%S)
```

```
BACKUP_DIR="/var/backups/project_db"
```

```
DB_NAME="Project_Database"
```

```
OUTPUT_FILE="$BACKUP_DIR/${DB_NAME}_$DATE.sql"
```

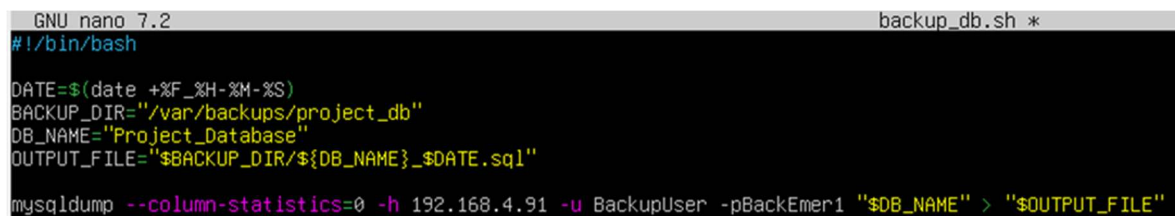
```
mysqldump --column-statistics=0 -h 192.168.4.91 -u BackupUser -pBackEmer1 "$DB_NAME" > "$OUTPUT_FILE"
```

-h is the database server IP

-u is the MySQL user

-pBackEmer1 is the password (no space after -p)

the output is then saved into a .sql file



```
GNU nano 7.2 backup_db.sh *
#!/bin/bash

DATE=$(date +%F_%H-%M-%S)
BACKUP_DIR="/var/backups/project_db"
DB_NAME="Project_Database"
OUTPUT_FILE="$BACKUP_DIR/${DB_NAME}_$DATE.sql"

mysqldump --column-statistics=0 -h 192.168.4.91 -u BackupUser -pBackEmer1 "$DB_NAME" > "$OUTPUT_FILE"
```

Execute the script using :

➔ `chmod +x ~/backup_db.sh`



```
glola041@glola041backup:/var/backups$ sudo chmod +x ~/backup_db.sh
glola041@glola041backup:/var/backups$ ./backup_db.sh
```



```
glola041@glola041backup:/var/backups$ ls
alternatives.tar.0  apt.extended_states.0  apt.extended_states.1.gz  dpkg.arch.0  dpkg.diversions.0  dpkg.statoverride.0  dpkg.status.0  project_db
```

Configure the shell script to run daily at 8:00 AM.

To automate the process, run `crontab -e` and alter the script to schedule cron to run every day at 8:00 a.m backup of the project database. Make the `backup_db.sh` script running to store on the backup server.

Add the following cron to run every day at 8:00 a.m:

➔ `0 8 * * * /home/glo1a041/backup_db.sh`

To test the cron configuration change temporarily the schedule to:

➔ `* * * * * /home/glo1a041/backup_db.sh`

This will run the script every minute, once confirmed it works change it back to the initial cron `0 8 * * *`.

```
GNU nano 7.2 /tmp/crontab.rtrRnZ/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
# * * * * * mysqldump -h 192.168.4.91 -u Project_Database > /var/backups/project_db/Backup_From_(date +%F_%H-%M-%S).sql
0 8 * * * /home/glo1a041/backup_db.sh
```

Set up the required FTP access

To make the backup files accessible remotely, FTP access was configured, some modifications were made to make sure to meet the requirement:

- ➔ anonymous_enable=NO
- ➔ local_enable=YES
- ➔ write_enable=YES
- ➔ chroot_local_user=NO

```
GNU nano 7.2 /etc/vsftpd.conf *
# Example config file /etc/vsftpd.conf
#
# The default compiled in settings are fairly paranoid. This sample file
# loosens things up a bit, to make the ftp daemon more usable.
# Please see vsftpd.conf.5 for all compiled in defaults.
#
# READ THIS: This example file is NOT an exhaustive list of vsftpd options.
# Please read the vsftpd.conf.5 manual page to get a full idea of vsftpd's
# capabilities.
#
# Run standalone? vsftpd can run either from an inetd or as a standalone
# daemon started from an initscript.
listen=NO
#
# This directive enables listening on IPv6 sockets. By default, listening
# on the IPv6 "any" address (:::) will accept connections from both IPv6
# and IPv4 clients. It is not necessary to listen on *both* IPv4 and IPv6
# sockets. If you want that (perhaps because you want to listen on specific
# addresses) then you must run two copies of vsftpd with two configuration
# files.
listen_ipv6=YES
#
# Allow anonymous FTP? (Disabled by default).
anonymous_enable=NO
#
# Uncomment this to allow local users to log in.
local_enable=YES
#
# Uncomment this to enable any form of FTP write command.
write_enable=YES
#
# Default umask for local users is 077. You may wish to change this to 022,
# if your users expect that (022 is used by most other ftpd's)
local_umask=022
#
# You may restrict local users to their home directories. See the FAQ for
# the possible risks in this before using chroot_local_user or
# chroot_list_enable below.
chroot_local_user=NO
```

Run:

- ➔ sudo systemctl restart vsftpd
- ➔ ./backup_db.sh
- ➔ cd /var/backups/project_db
- ➔ ftp 192.168.4.56
- ➔ Ls

Atelier

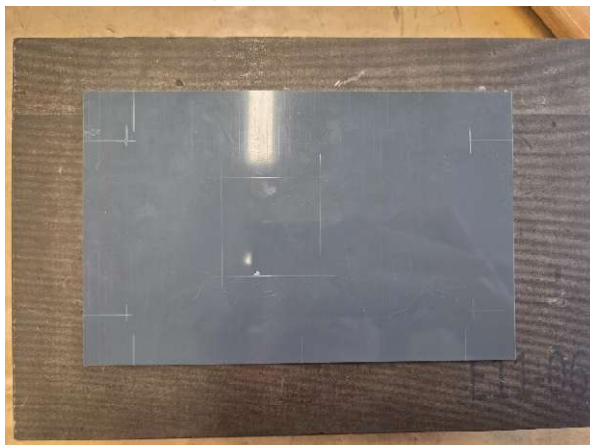
A1 – PVC holder

Step one: pin pointing

I'm using a height gauge, a precision measuring instrument for marking out and inspection.



PVC display holder



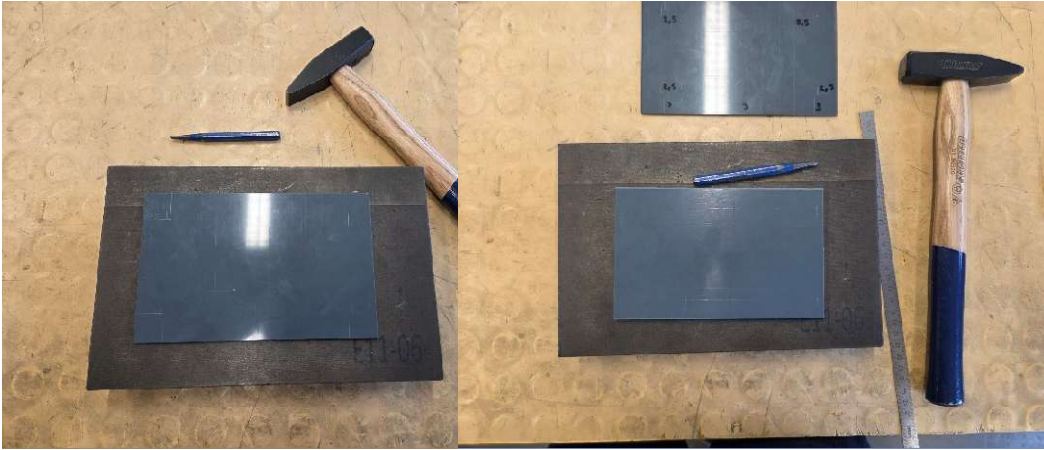
PVC base plate



Where you want to drill the hole you punch a mark with hexagonal center punch, a tool used to mark a fixed point on materials. And write up hole size with a marker.

PVC display holder

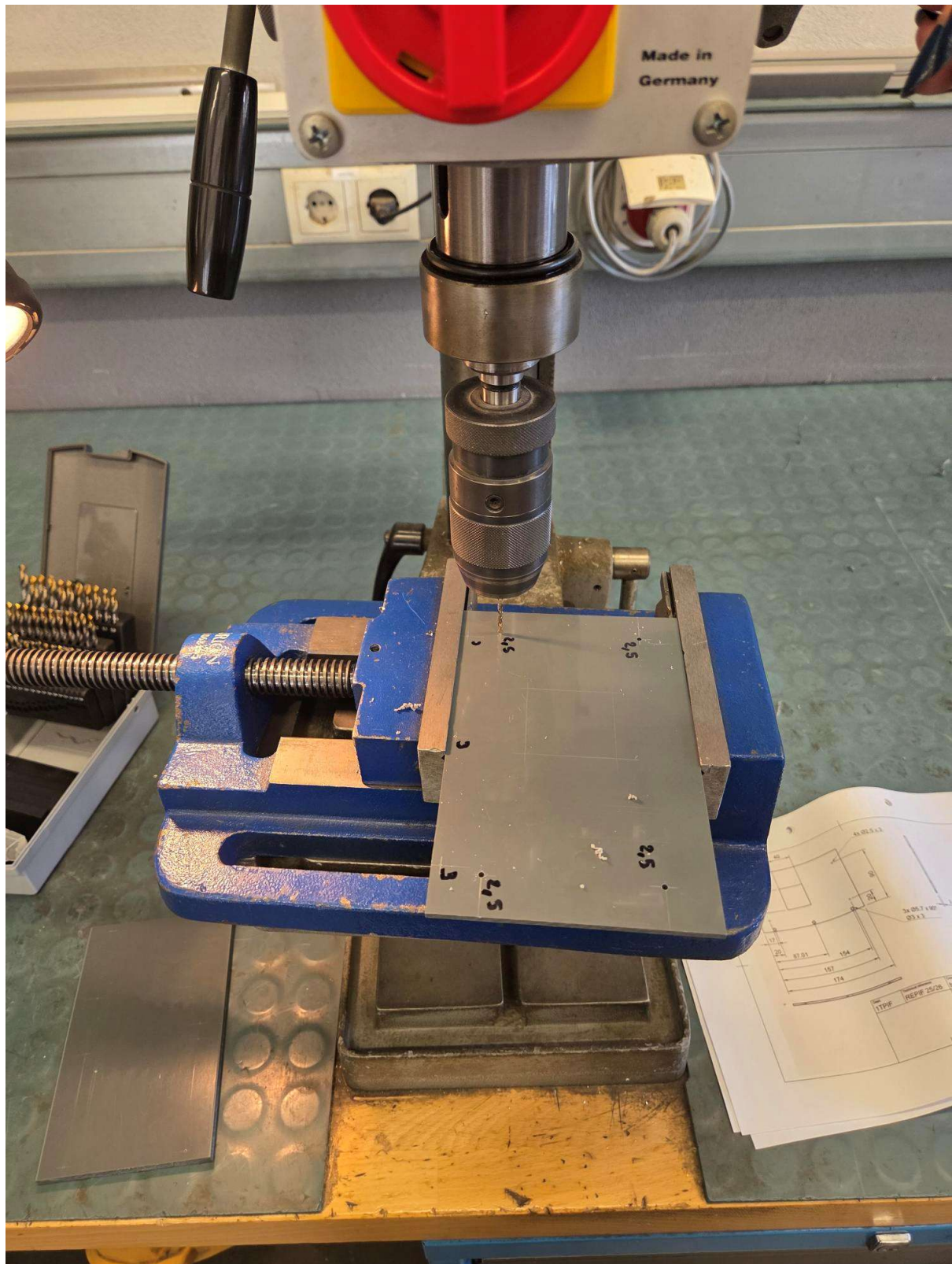
PVC base plate





Step two : Drilling

Using the bench drill, drill a hole at each mark following the hole size you marked earlier on your plate or the **technical drawing provided**.



For the PVC base plate, on the side drill first a **2,5 Diameter**, then use a **thread tap (metric M3 tap with a 0.5 mm thread pitch)**

Error: During the process the thread tap got stuck in the plate, had to turn it out.

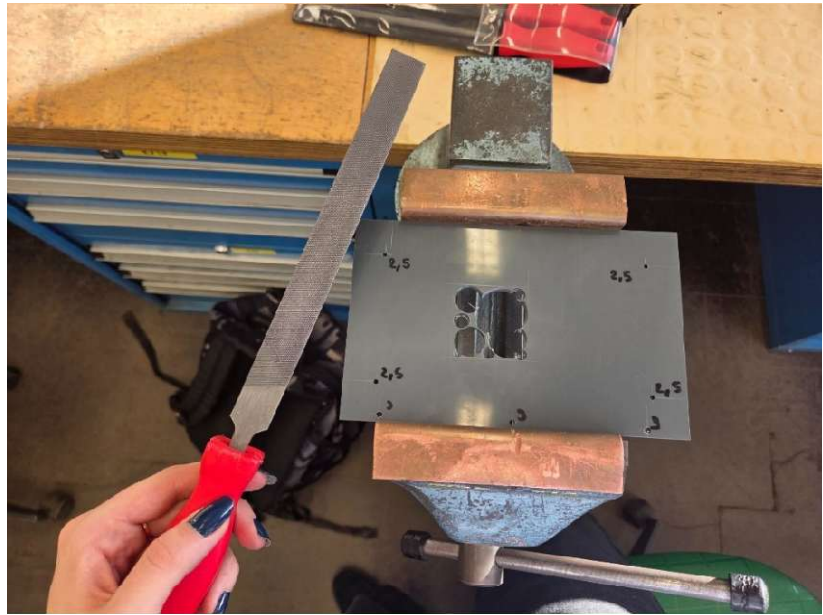


For the PVC display holder drill multiple holes to make it easier to make the rectangle.



Step 3: Cleaning up

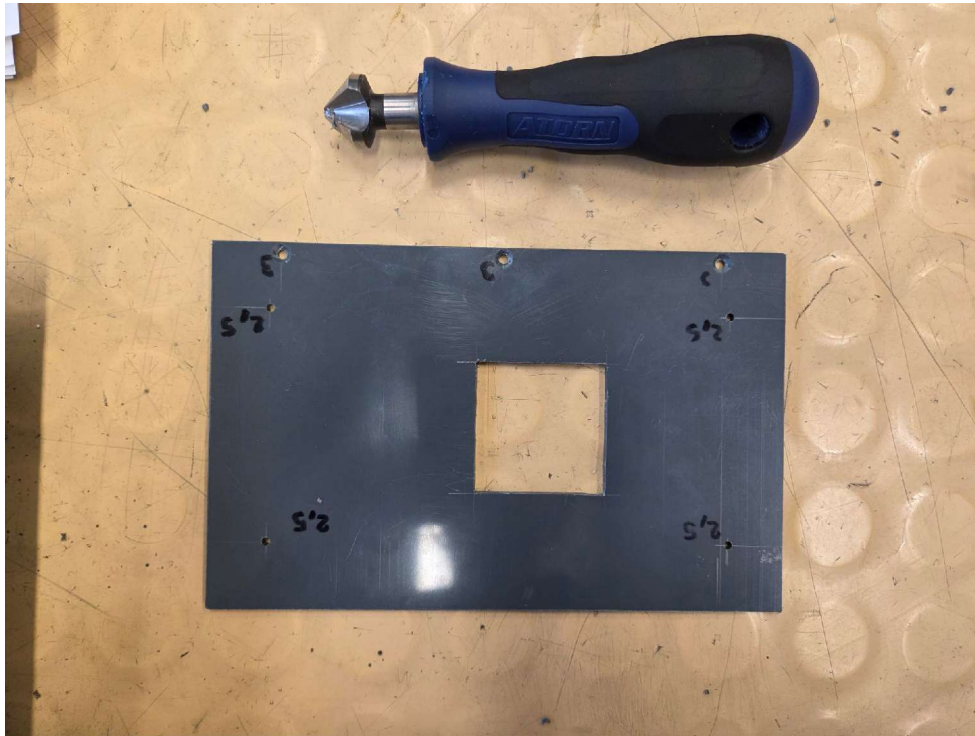
Using a **workshop file**, **trim the** PVC display holder to give it the rectagle shape wanted.



Preferably to not cut yourself handling the plate, trim the sides of both plates.



Using the hand countersink, following the **technical drawing provide**, so the screws will tighten directly into this plate.

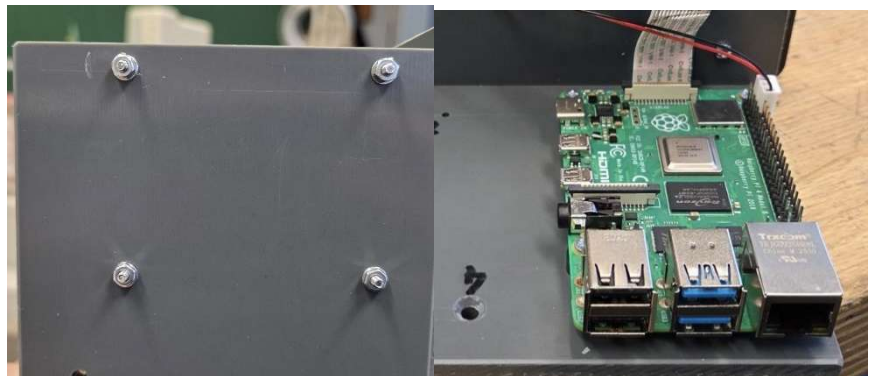


Step four: Assembling

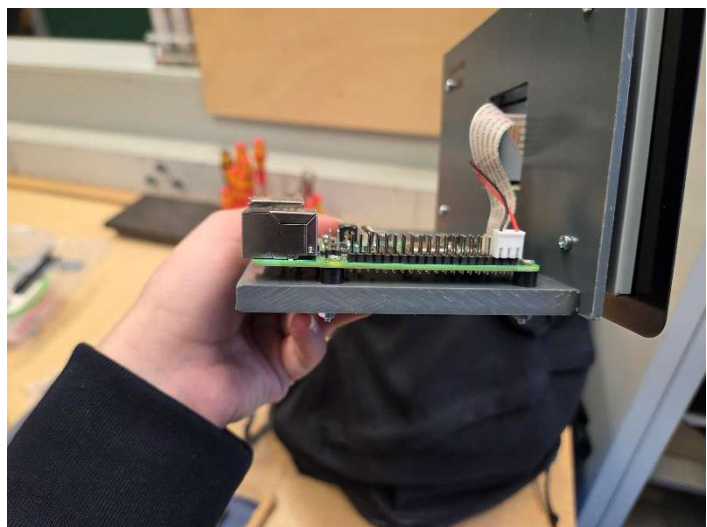
Using the screw provided, screw the two plates together.



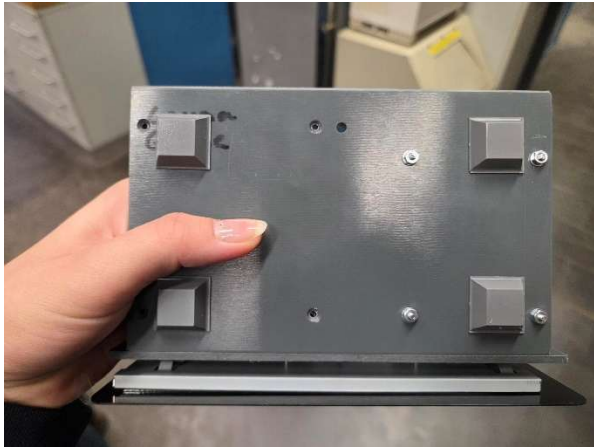
Detach the raspberry pi, from the screen and screw the raspberry pi to the PVC base plate.



One PVC display holder screw the screen and plug the cables in the raspberry pi.

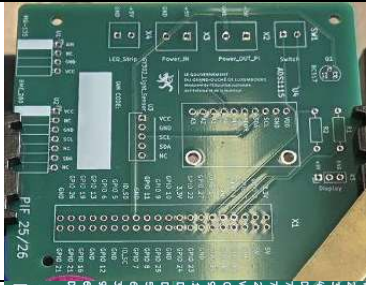
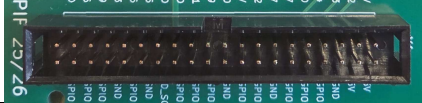
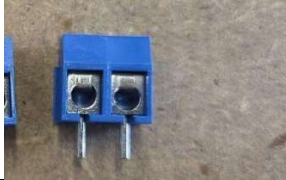
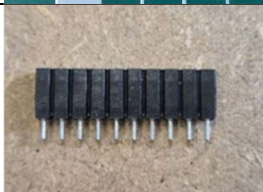


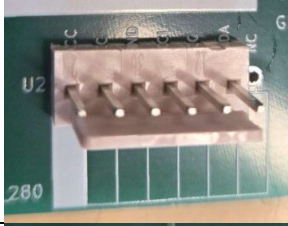
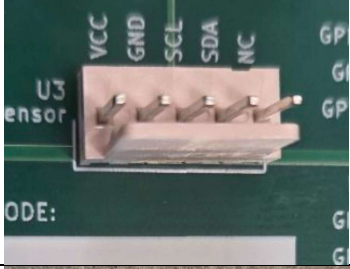
Lastly on the PVC base plate put the rubber mounting studs, and you're done.



A2 – Printed circuit board

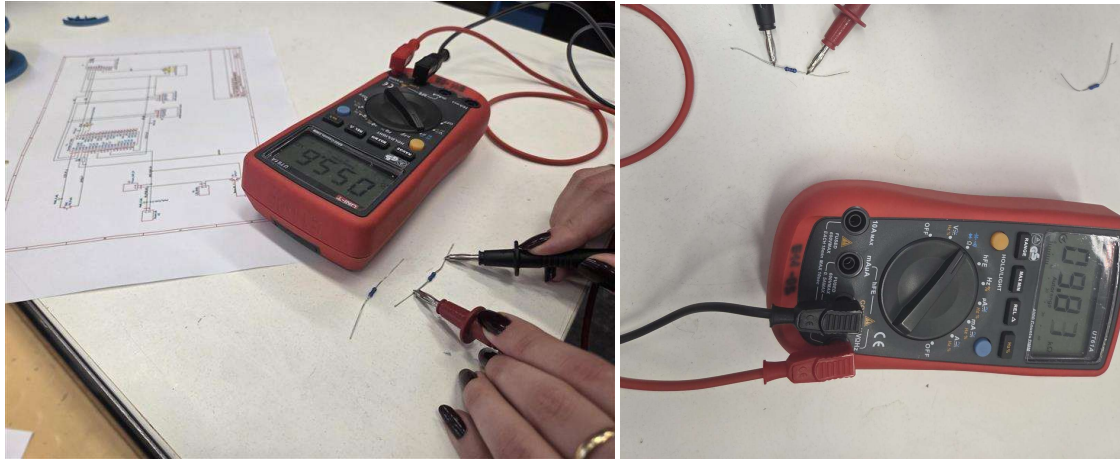
List of material

Component	Quantity	
2-layer printed circuit board 75 x 85 mm	1	
40-pin straight connector strip R WS40G	1	
Metal resistor 0207 0.6W 10K	1	
Metal resistor 0207 0.6W 56K	1	
BC517 transistor	1	
PCB connection terminal 2-pin RM 5	2	
PCB terminal block, 2-pin RM 3.5	2	
1-row angled socket strip 1 x 4 -RM2.54mm h=8mm	1	
1-row angled socket strip 1 x 10 - RM2.54mm h=8mm		

1-row angled female connector strip 1 x 7 - RM2.54mm h=8mm	1	
1-row angled female connector strip 1 x 5 - RM2.54mm h=8mm	1	
Pin strip, 1 row, straight, 1 x 3 - RM2.54 mm	1	
ADS1115 ADC module I2C	1	
Pin strip, 1 row, straight, 1 x 10 - RM2.54 mm	1	

Step 1: Verify Resistors

To verify both resistors, check them with a multimeter. One resistor - **Metal resistor 0207 0.6 W 10 k** - should measure approximately **10 k Ω** , and the other - **Metal resistor 0207 0.6 W 56 k** - should measure approximately **56 k Ω** .

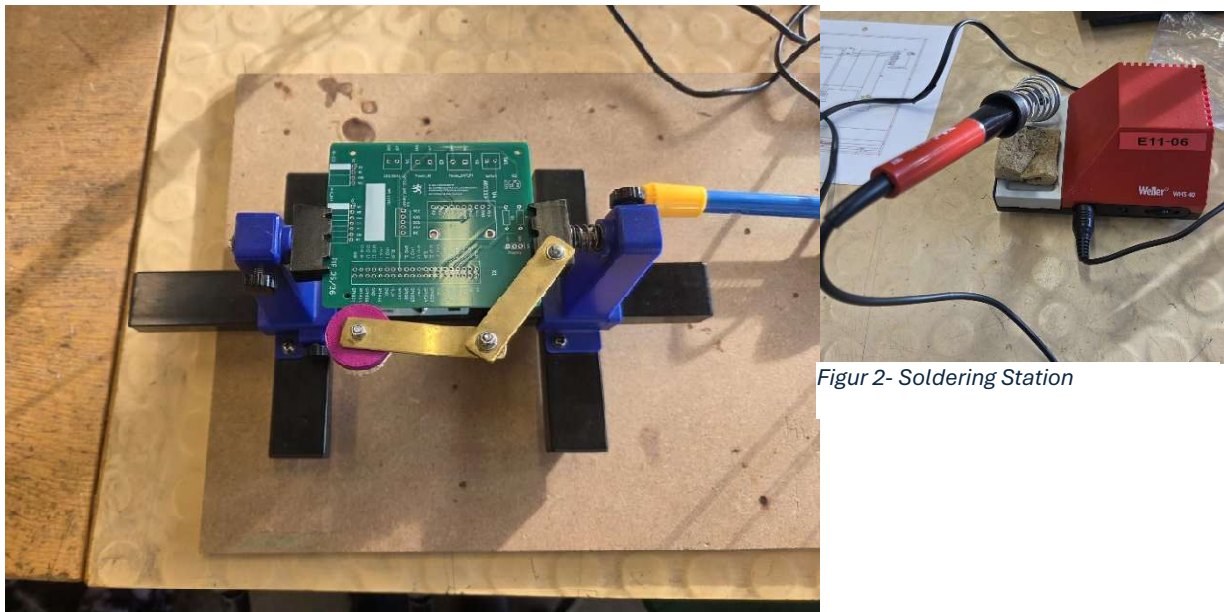


Figur 1 Multimeter

Step 2: Prepare for Soldering

For more comfort during the soldering part put the Printed circuit board on an Adjustable Pcb holder. Turn on your **soldering station**.

Tip: Always start soldering with the smallest components first.

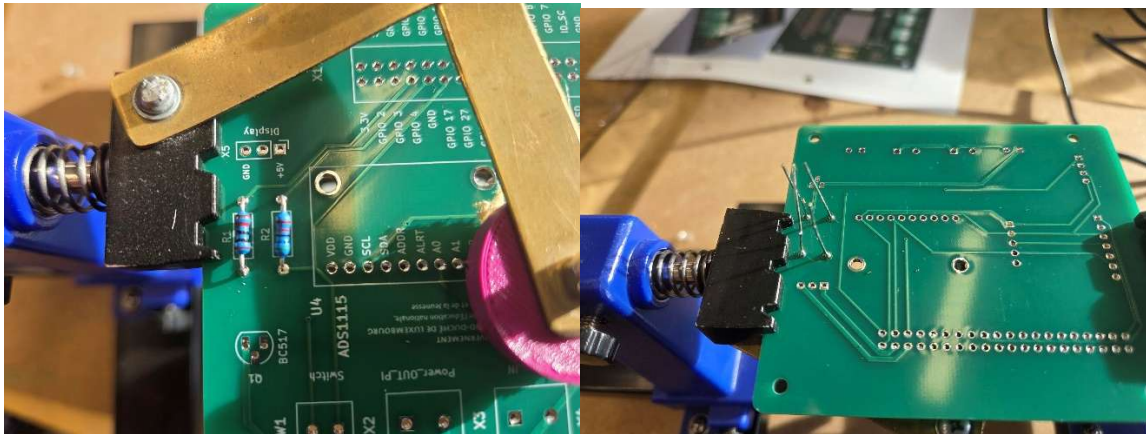


Figur 2- Soldering Station

Figur 3- Adjustable Pcb holder

Step 3: Solder Resistors

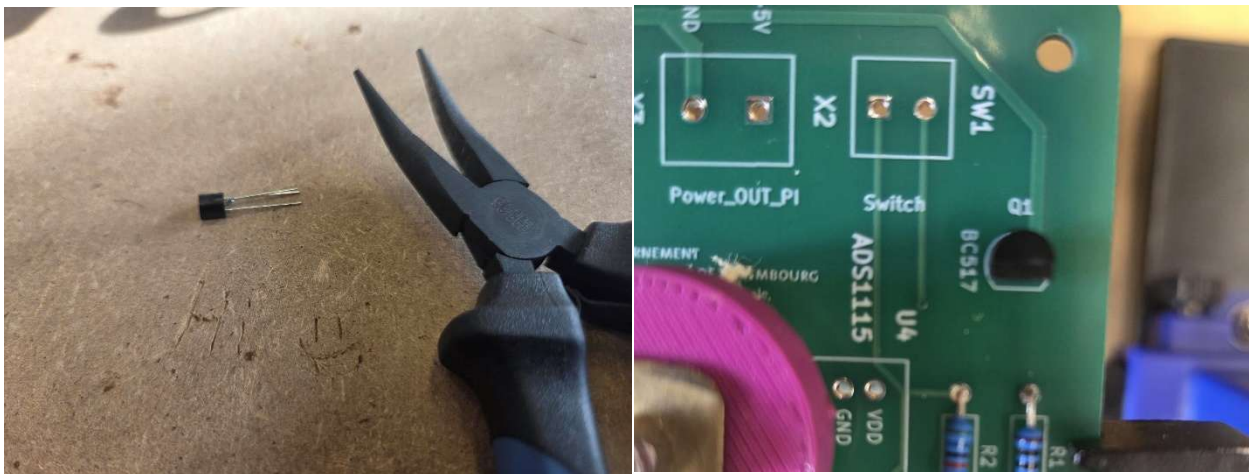
Solder the **10 k** into **R1** and the **56 k** in the **R2**. Cut off the excess legs after soldering.



Figur 4 and 5 - resistor soldered

Step 4: Solder the Transistor

Before Bend the legs of the **BC517** transistor to fit the holes, then solder it into **Q1** and cut off the excess legs.



Figur 6 - transistor bended legs

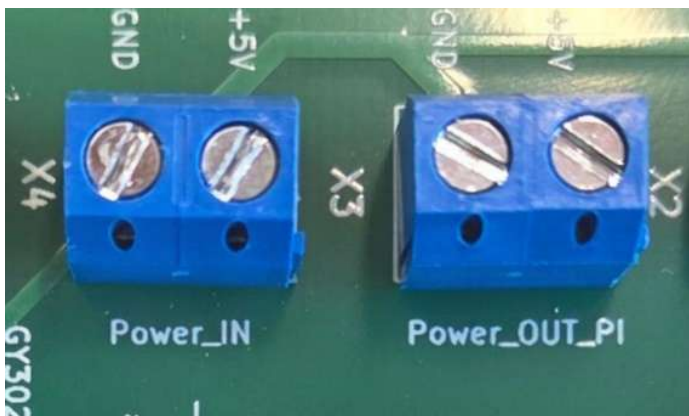
Figur 5 - transistor front

Step 5: Solder Terminal Blocks

Solder both PCB terminal block, 2-pin RM 3.5 on SW1 respectively on switch and LED_Strip.



Figur 7 and 9 - PCB terminal block soldered

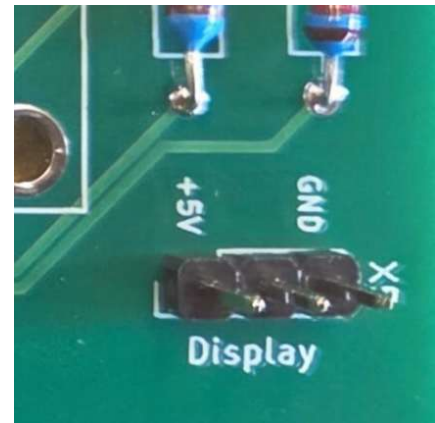


Figur 10 - PCB connection terminal soldered

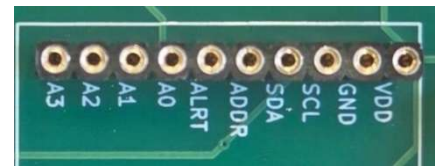
Solder both PCB connection terminal 2-pin RM 5 onto **Power_IN** and **Power_OUT_PI**, leaving the legs.

Step 6: Solder Pin and Socket Strips

On **X5**, solder a Pin strip, 1 row, straight, 1 x 3 - RM2.54 mm



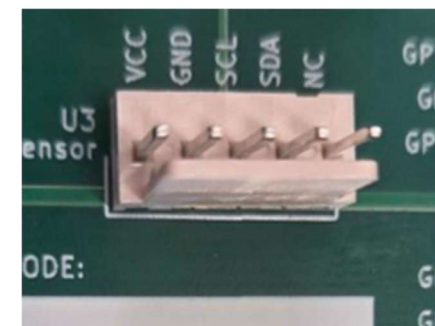
On **U4**, solder 1-row angled socket strip 1 x 10 -RM2.54mm h=8mm.



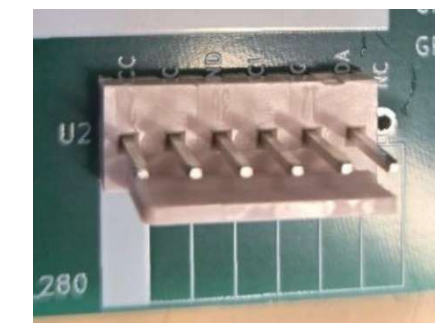
On **U1**, solder 1-row angled socket strip 1 x 4 -RM2.54mm h=8mm.



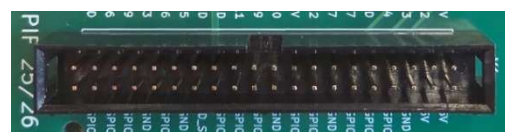
On **U3**, solder a 1-row angled female connector strip 1 x 5 - RM2.54mm h=8mm



On **U2**, solder a 1-row angled female connector strip 1 x 7 - RM2.54mm h=8mm

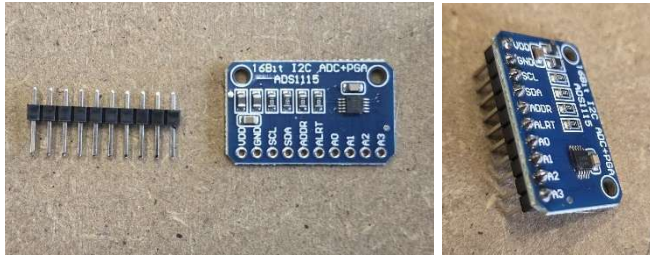


On **X1**, solder the 40-pin straight connector strip R WS40G

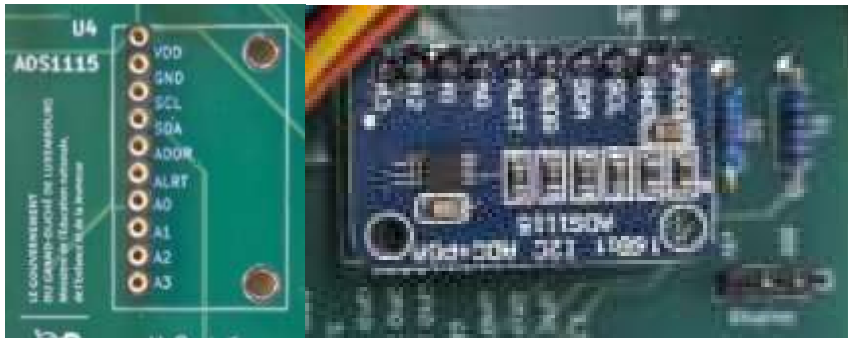


Step 7: Connect ADS1115 ADC Module

Solder a Pin strip, 1 row, straight, 1 x 10 - RM2.54 mm to the ADS1115 ADC module I2C.



Insert it into the 1-row angled socket strip 1 x 10 -RM2.54mm h=8mm already on U4.

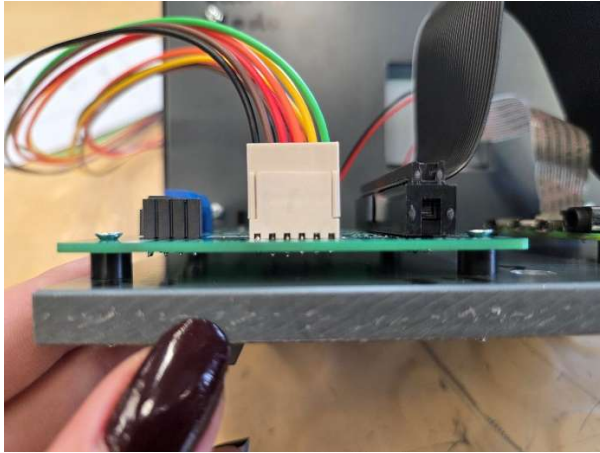


Figur 11 - Indication of U4

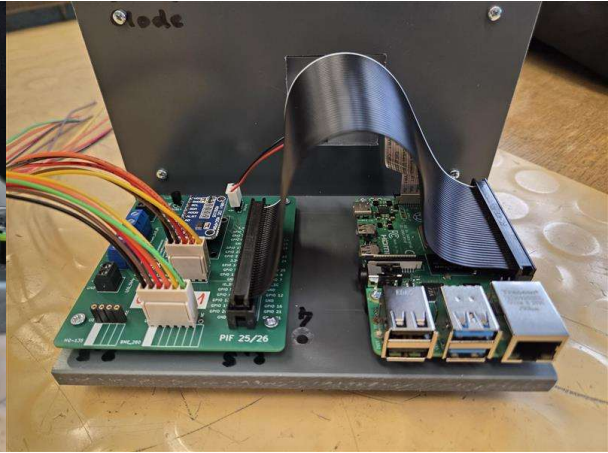
Figur 12 - ADS1115 inserted in the socket

Step 8: Mount the circuit board on the PVC plate.

Space the PCB **5 mm above the PVC plate** using spacers. And mount the cables.



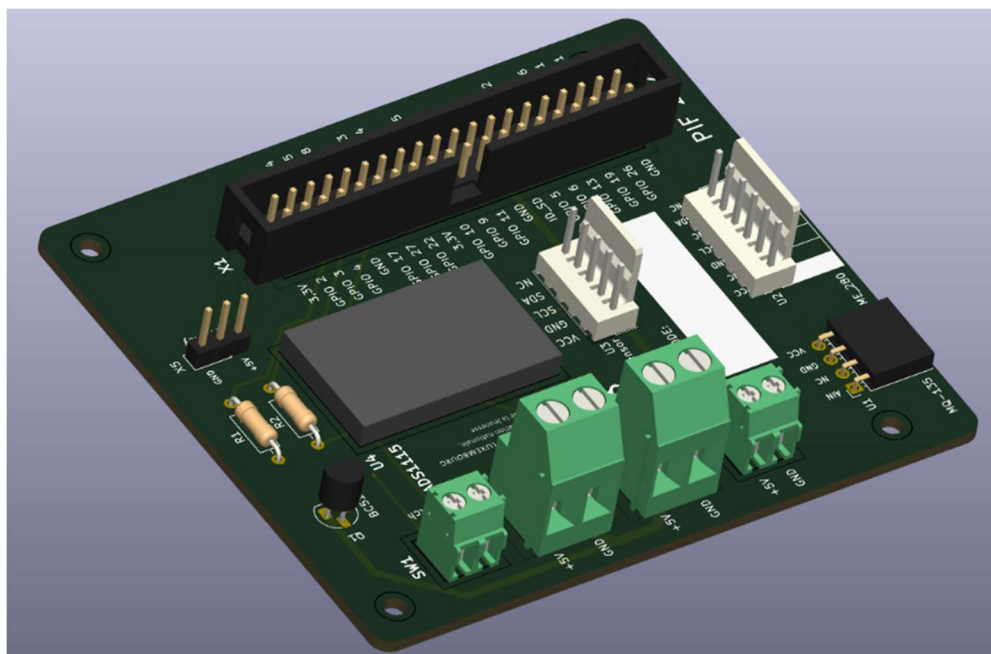
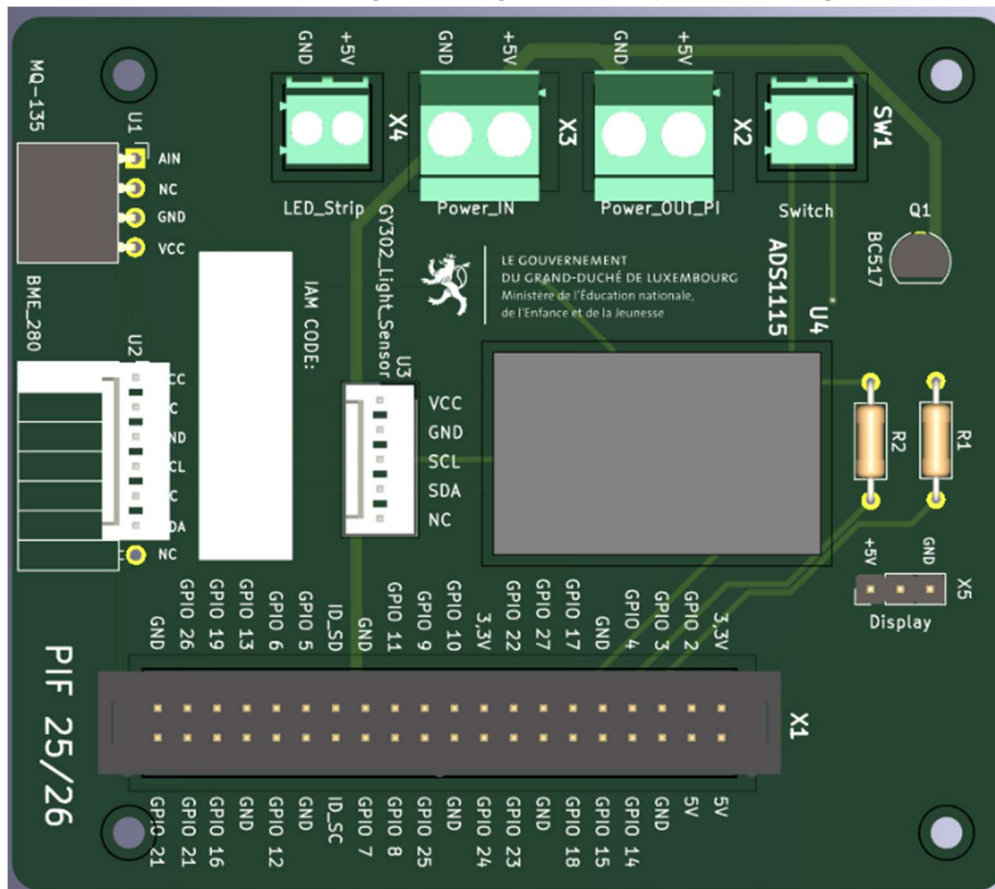
Figur 12 - 5 mm spacers



Figur 13 - Whole mounted project

Overall view

There is a more accurate image showing which component belongs where.

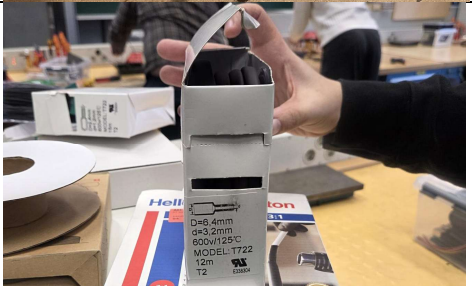


Mistakes

I had a problem with one of the 1-row angled female connectors. It wasn't sitting straight on the PCB, but it was already soldered. I ended up using two soldering stations to quickly adjust and solder it straight.

A3 – Power Supply

List of material

Component	Quantity	
Cable Insulation	1	
Heat Shrink Tubin	15 cm	
Heat Gun	1	
Soldering station	1	

Solder	3 m	
Cable stripper	2 different sizes	
Diagonal cutters	1	
Automatic wire stripping pliers	1	

Information:

This is a practical wire repair.

Step one: Cutting and Stripping the Insulation

1. **Cut the Cable Insulation:**

Use diagonal cutters to cut the Cable Insulation in two pieces without damaging the internal wires.



2. **Strip the First Layer:**

Use a larger cable stripper to remove the first layer of insulation, stripping about **7 cm**.

3. **Roll Up the Grey Fiber:**

Inside, you will find a grey fiber. Roll it up and set it aside.



4. **Strip the Second Layer:**

Use a smaller cable stripper to remove the next layer, exposing the internal wires for reconnection



5. **Strip the Last Protection:**

Use automatic wire stripping pliers to remove the final layer of protection, exposing the wires underneath for reconnection.



S

Step two: Preparing and Soldering the Wires

1. Cut Each Wire to Length:

Cut each wire to match the length of the cable it will be connected to.

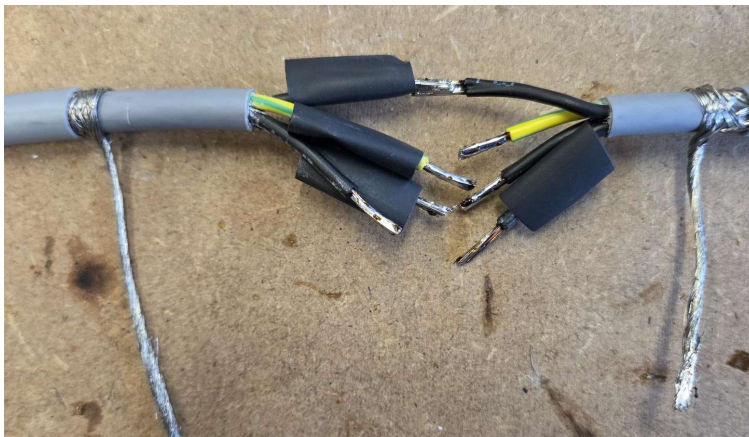


2. Soak the Wires in Solder:

Using a soldering iron, heat the exposed wires and apply solder to ensure a solid connection. Do the same for the Grey Fiber.

3. Prepare for Soldering:

Slide heat shrink tubing onto the wires and Grey Fiber before soldering them.



4. Solder the Wires Together:

Solder the wires and Grey Fiber together to form a secure electrical connection.



Step three: Insulating

1. Position the Heat Shrink Tubing:

Slide the tubing over the soldered area so it fully covers the wire and Grey Fiber.

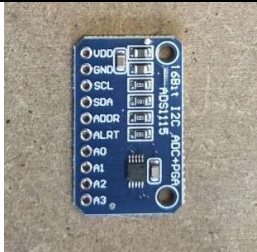
2. Apply Heat:

Use a heat gun to shrink the tubing tightly around the wire and Grey Fiber.



A4 – Sensor #1

List of material

Component	Quantity	
MQ-series Gas Sensor	1	
ADS1115 ADC module I2C	1	
Pin strip, 1 row, straight, 1 x 10 - RM2.54 mm	1	

Step one: Connect ADS1115 ADC Module

Solder a Pin strip, 1 row, straight, 1 x 10 - RM2.54 mm to the ADS1115 ADC module I2C.

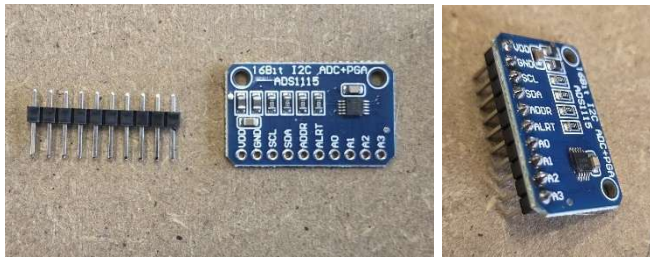


Figure 2- ADS1115 ADC module I2C and Pin strip

Figure 1 - Pin strip and ADS1115 soldered together

Insert the ADS1115 into the 1-row angled socket strip 1 x 10 -RM2.54mm h=8mm already on U4.

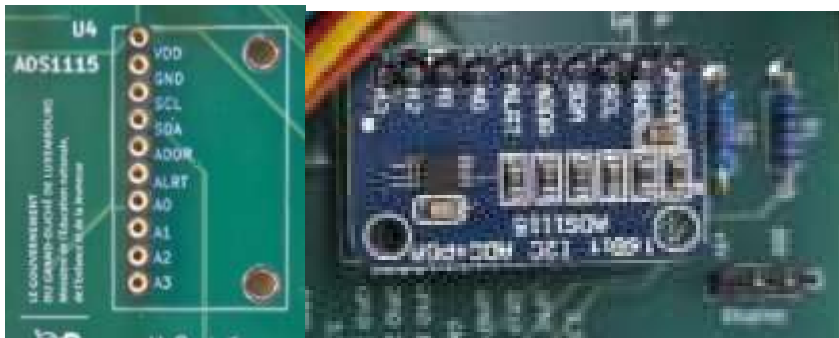


Figure 3- ADS1115 inserted in the socket

Figure 4 - Indication of U4

Step two: Connect Gas Sensor

Insert the Gas sensor into the 1-row angled socket strip 1 x 4 on the U1.

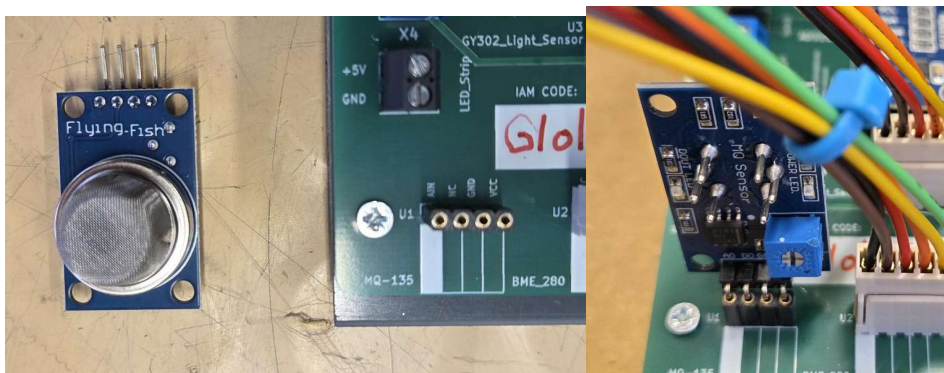


Figure 5 - Gas sensor inserted to PCB

Figure 6 - Gas sensor and Indication of U1

Step three: Raspberry configuration

Using terminal commands to verify communication with the connected sensor enter:

➔ `i2cdetect -y 1`

To detect all connected devices and confirm their addresses.

```
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
10:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
20:  --  --  --  23  --  --  --  --  --  --  --  --  --  --  --  --
30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
40:  --  --  --  --  --  --  --  48  --  --  --  --  --  --  --  --
50:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
60:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
70:  --  --  --  --  --  --  76  --
```

There it confirms the presence of the ADS1115 and the Gas sensor at the address 0x48.

To verify communication with the ADS1115, a test was made.

➔ `i2cget -y 1 0x48 0x00 w`

```
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ i2cget -y 1 0x48 0x00 w
0x0000
```

Change the address of the test-mq135.py to the correct I²C address (0x48).

➔ `cd /opt/station/Firmware und Scripts`

➔ `sudo nano test-mq135.py`

```
ADDRESS = 0x48 # TODO: Change this

from mq135 import MQ135
import smbus
def main():
    bus = smbus.SMBus(1) # Use 1 for Raspberry Pi, 0 for older models
    sensor = MQ135(bus, ADDRESS)

    try:
        gas = sensor.read_CO2()
        print(f"CO2 concentration: {gas} ppm")
    except Exception as e:
        print(f"Error reading CO2 concentration: {e}")

if __name__ == "__main__":
    main()
```

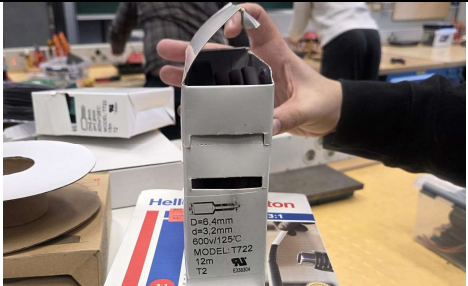
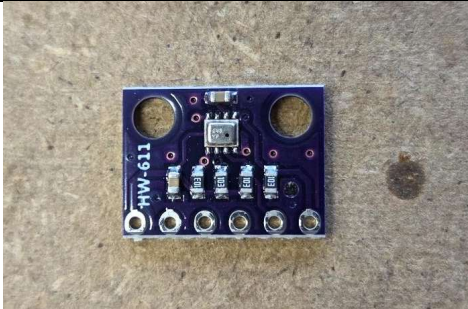
Finally, the sensor output was tested using the following command:

➔ `python3 test-mq135.py`

```
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ python3 test-mq135.py
CO2 concentration: 2141 ppm
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $
```

A5 – Sensor #2

List of material

Component	Quantity	
Heat Gun	1	
Soldering station	1	
Solder	1 m	
Heat Shrink Tubin	2 cm	
Enviromental sensor module	1	

Light sensor module	1	
JST 4-pin connector cable	1	
JST 5-pin connector cable	1	
Diagonal cutters	1	

Step one: Sensor Installation and Cabling

The environmental sensor module and the light sensor module are connected using different coloured wires to clearly identify power and signal connections. The pins were soldered onto the board according to the instructions to ensure a secure connection.

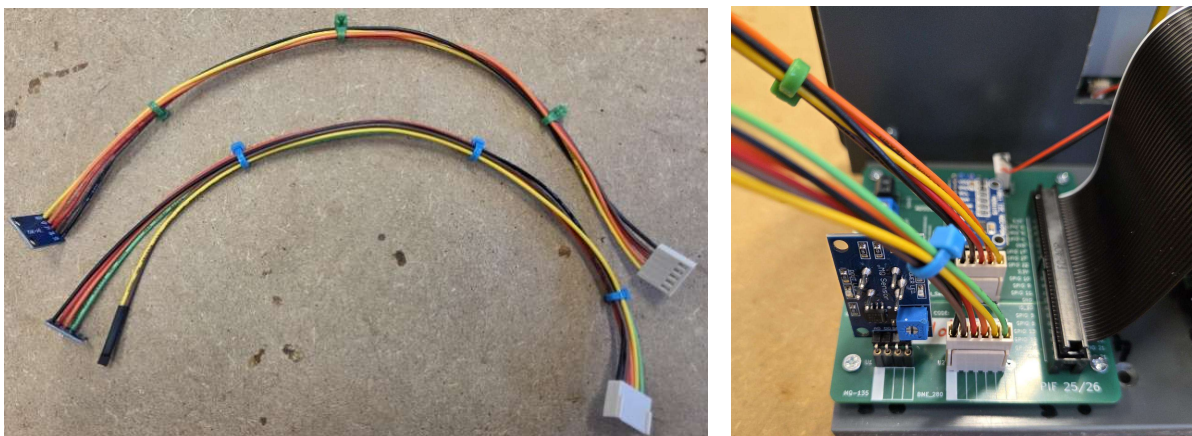
The light sensor is connected to VCC, GND, SCL, SDA, and ADDR. The Environmental sensor supplied is connected to the ports to VCC, GND, SCL, and SDA. The two ports CSB and SDO are not connected.



The CSB and SDO wire ends were cut off and insulated using heat shrink tubing to prevent short circuits.



The cables were organized with cable ties for cable management and inserted into the environmental sensor module and the light sensor module on the Printed circuit board.



Step two: Raspberry configuration

Using terminal commands to verify communication with the connected sensor enter:

➔ `i2cdetect -y 1`

To detect all connected devices and confirm their addresses.

```
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
10:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
20:  --  --  --  23  --  --  --  --  --  --  --  --  --  --  --  --
30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
40:  --  --  --  --  --  --  --  48  --  --  --  --  --  --  --  --
50:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
60:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
70:  --  --  --  --  --  --  76  --  --  --  --  --  --  --  --  --
```

There it confirms the presence of the environmental sensor module at the address 0x76 and the light sensor module at the address 0x23.

Change the address of the test-bh1750.py to the correct I²C address (0x48).

➔ `cd /opt/station/Firmware und Scripts`

➔ `sudo nano test-bh1750.py`

```
ADDRESS = 0x23 # TODO: Change this

from bh1750 import BH1750
import smbus
def main():
    bus = smbus.SMBus(1) # Use 1 for Raspberry Pi, 0 for older models
    sensor = BH1750(bus, ADDRESS)

    try:
        light_level = sensor.read_light()
        print(f"Light Level: {light_level} lux")
    except Exception as e:
        print(f"Error reading light level: {e}")

if __name__ == "__main__":
    main()
```

Test the light sensor output using the following command:

➔ `python3 test-bh1750.py`

```
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ python3 test-bh1750.py
Light Level: 42.5 lux
```


Because the environmental sensor module was changed, a new Python library had to be installed, and a new test script was created to communicate with the sensor via the I²C interface.

Install the new library and create the virtual environment called venv using the following command:

- ➔ `sudo apt install -y python3-venv python3-full`
- ➔ `python3 -m venv venv`

```
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ sudo apt install -y python3-venv python3-full
python3-venv is already the newest version (3.13.5-1).
python3-full is already the newest version (3.13.5-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 186
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ python3 -m venv venv
```

Activate the virtual environment, install I²C tools and install the Python library for the BMP280 sensor using the following command:

- ➔ `source venv/bin/activate`
- ➔ `sudo apt install -y python3-smbus i2c-tools`
- ➔ `pip install bmp280`

```
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ source venv/bin/activate
(venv) glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ sudo apt install -y python3-smbus i2c-tools
python3-smbus is already the newest version (4.4-2).
i2c-tools is already the newest version (4.4-2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 186
(venv) glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ pip install bmp280
Collecting bmp280
  Downloading bmp280-1.0.0-py3-none-any.whl.metadata (3.8 kB)
Collecting i2cdevice>=1.0.0 (from bmp280)
  Downloading i2cdevice-1.0.0-py3-none-any.whl.metadata (7.3 kB)
Collecting smbus2 (from i2cdevice>=1.0.0->bmp280)
  Downloading smbus2-0.6.0-py2.py3-none-any.whl.metadata (7.2 kB)
Downloaded bmp280-1.0.0-py3-none-any.whl (7.2 kB)
Downloaded i2cdevice-1.0.0-py3-none-any.whl (10 kB)
Downloaded smbus2-0.6.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: smbus2, i2cdevice, bmp280
Successfully installed bmp280-1.0.0 i2cdevice-1.0.0 smbus2-0.6.0
```

Change the test-280.py file.

- ➔ `sudo nano test-bme280.py`

```
from bmp280 import BMP280
from smbus2 import SMBus
import time
```

```
ADDRESS = 0x76
```

```
def main():
    bus = SMBus(1)
    sensor = BMP280(i2c_dev=bus, i2c_addr=ADDRESS)

    while True:
        temperature = sensor.get_temperature()
        pressure = sensor.get_pressure()

        print(f"Temperature: {temperature:.2f} °C")
        print(f"Pressure: {pressure:.2f} hPa")
        print("-" * 30)
        time.sleep(2)
```

```
if __name__ == "__main__":
    main()
```

```
GNU nano 8.4 test-bme280.py
from bmp280 import BMP280
from smbus2 import SMBus
import time

ADDRESS = 0x76

def main():
    bus = SMBus(1)
    sensor = BMP280(i2c_dev=bus, i2c_addr=ADDRESS)

    while True:
        temperature = sensor.get_temperature()
        pressure = sensor.get_pressure()

        print(f"Temperature: {temperature:.2f} °C")
        print(f"Pressure: {pressure:.2f} hPa")
        print("-" * 30)
        time.sleep(2)

if __name__ == "__main__":
    main()
```

Test the environment sensor output in the virtual environment using the following command:

➔ `venv/bin/python3 test-bmp280.py`

```
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ venv/bin/python3 test-bme280.py
Temperature: 23.90 °C
Pressure: 994.85 hPa
-----
Temperature: 23.90 °C
Pressure: 994.84 hPa
-----
Temperature: 23.90 °C
Pressure: 994.84 hPa
-----
Temperature: 23.90 °C
Pressure: 994.84 hPa
-----
Temperature: 23.90 °C
Pressure: 994.86 hPa
-----
```

Install the package outside the virtual environment so that the bmp280 can be used globally instead of inside venv using the following command:

➔ `python3 -m pip install bmp280 --break-system-packages`

```
glola041@raspberrypiglola041:/opt/station/Firmware und Scripts $ python3 -m pip install bmp280 --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Collecting bmp280
  Using cached bmp280-1.0.0-py3-none-any.whl.metadata (3.8 kB)
Collecting i2cdevice>=1.0.0 (from bmp280)
  Using cached i2cdevice-1.0.0-py3-none-any.whl.metadata (7.3 kB)
Requirement already satisfied: smbus2 in /usr/lib/python3/dist-packages (from i2cdevice>=1.0.0->bmp280) (0.4.3)
Using cached bmp280-1.0.0-py3-none-any.whl (7.2 kB)
Using cached i2cdevice-1.0.0-py3-none-any.whl (10 kB)
Installing collected packages: i2cdevice, bmp280
Successfully installed bmp280-1.0.0 i2cdevice-1.0.0
```

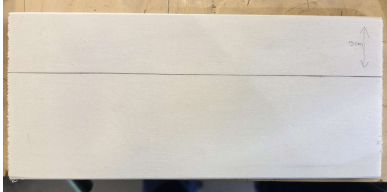
Test the environment sensor output using the following command:

➔ `python3 test-bmp280.py`

```
glola041@raspberrypi glola041:/opt/station/Firmware und Scripts $ python3 test-bme280.py
Temperature: 25.21 °C
Pressure: 995.82 hPa
-----
Temperature: 25.22 °C
Pressure: 995.83 hPa
-----
```


A6 - Assembly

List of material

Component	Quantity	
Base plate	1	
Right frame	1	
Left frame	1	
Plastic dowel pin	2	
Rear panel	1	

Removable cover	1	
Push button	1	

Step one: Drilling the hole for the push button

The position for the push button was measured and marked on the left frame. A hole was then drilled so that the push button can fit properly. (50 mm from the left and 80 mm from the top)

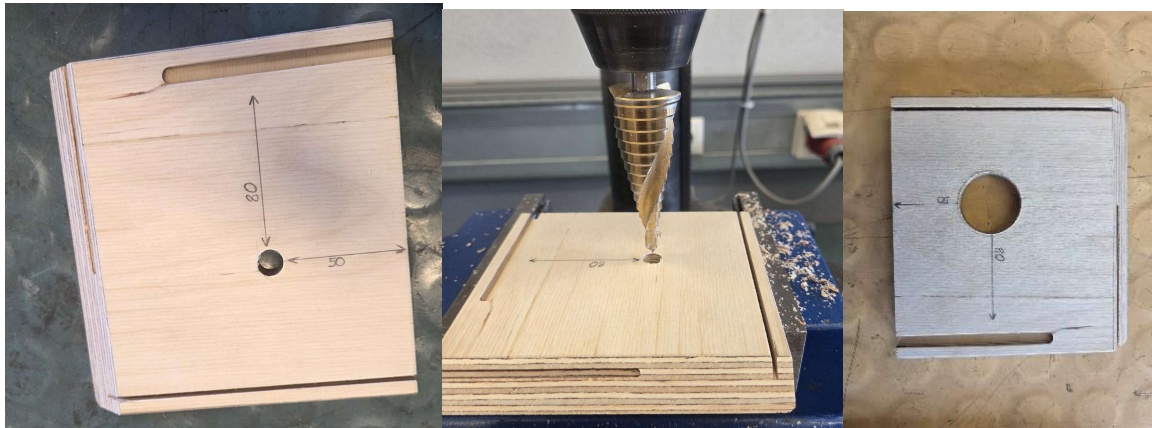


Figure 7- Left frame

Figure 8 - Left frame drilling

Figure 9 - Left frame drilled

Step two: Rear panel preparation drilling sensor mounting

The position for the sensors was marked (3 cm from the top) on the rear panel. A hole was then drilled at the marked points.

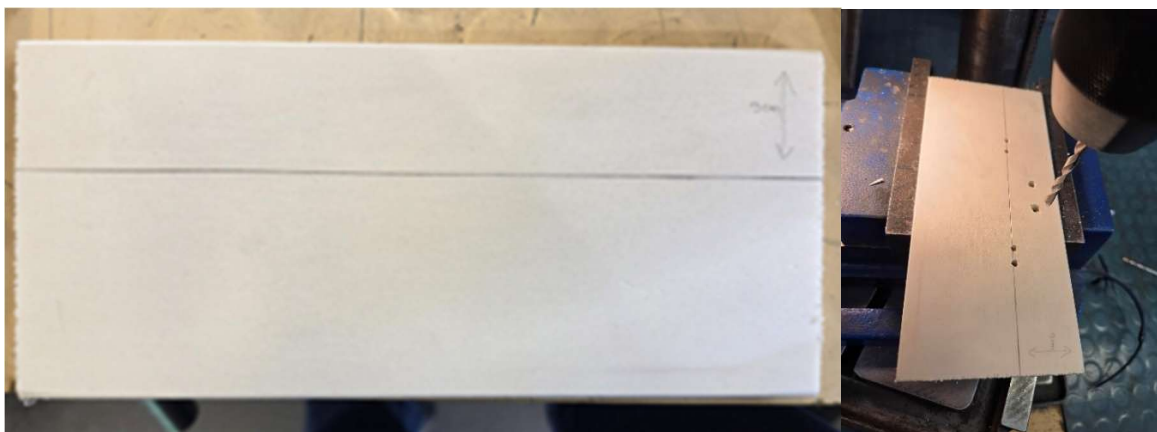


Figure 10 - Rear panel measured

Figure 11 - Rear panel drilling

Mount the light sensor and barometric sensor on the rear breadboard of the housing.

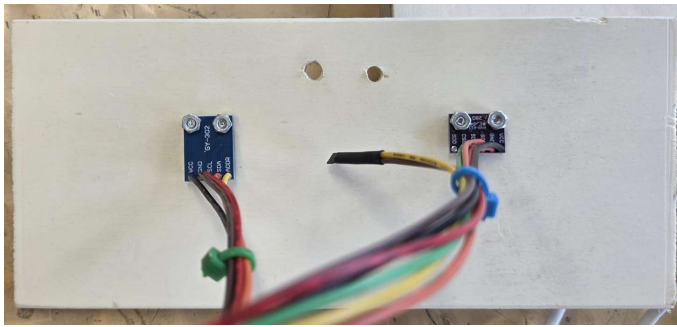


Figure 12 - Light sensor and barometric sensor mounted

Step three: Assembling the frame and base

Plastic dowel pins were inserted into the base plate, right frame, and left frame to secure them together



Figure 13 - Assembled frame and base

Step four: Installing the push button

I removed the insulation of the red and black wires using a wire-stripping tool then I soldered the wires to the push button terminals (two pins with the plastic base). The push button was then installed into the hole in the left frame and connected to the SW1 on the printed circuit board.

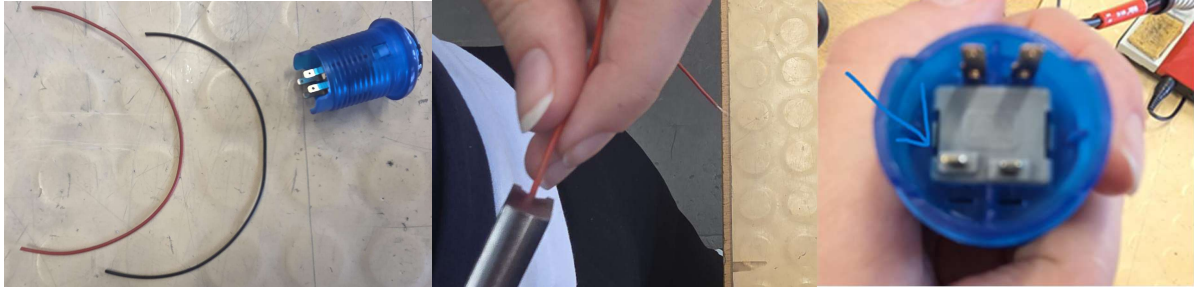


Figure 14 - Red and black wires and push button

Figure 15 - Removal of the insulation

Figure 16 - Two pins with the plastic base

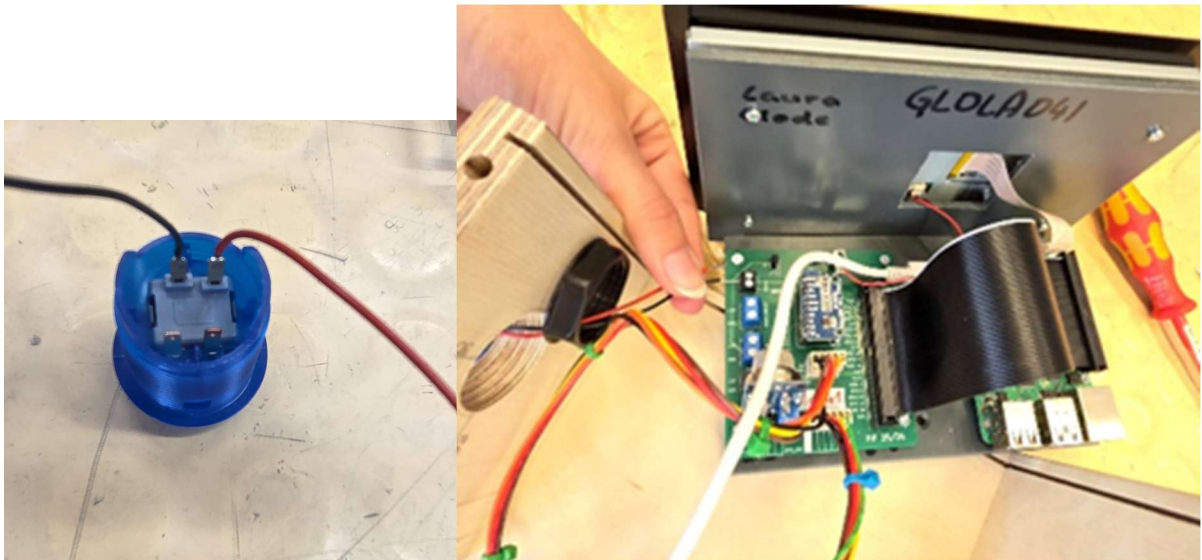


Figure 17- Connection to the SW1

Figure 18- Soldered bush button

Step five: Final assembly

The station was placed inside the wooden housing and placed correctly on the base plate. The rear panel mounded with the sensors was placed with the frame.

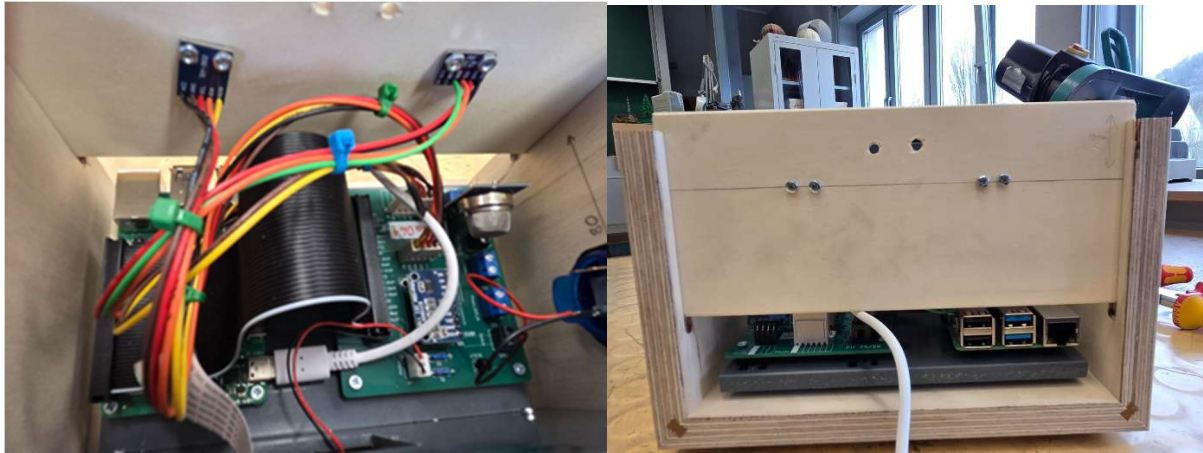


Figure 19 - Inside final assembly

Figure 20 - Rear panel final assembly

Difficulties

During the installation of the push button, I confused the the Power_OUT_PI on the printed circuit board to be the output. This error was rectified by reconnecting the push button to the SW1.

A7 - Lighting

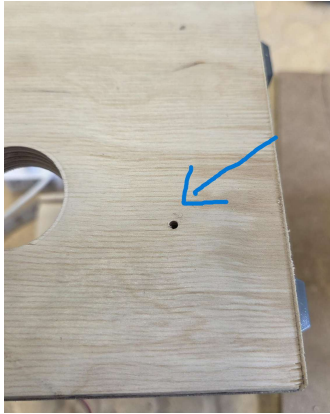
List of material

Component	Quantity	
Cables	2 x =< 15 cm	
LED strip	1 x 17 cm	
Soldering station	1	
Solder	20 cm	
Wire stripper	1	
Diagonal cutters	1	

Glue gun	1	
Hand Drill	1	

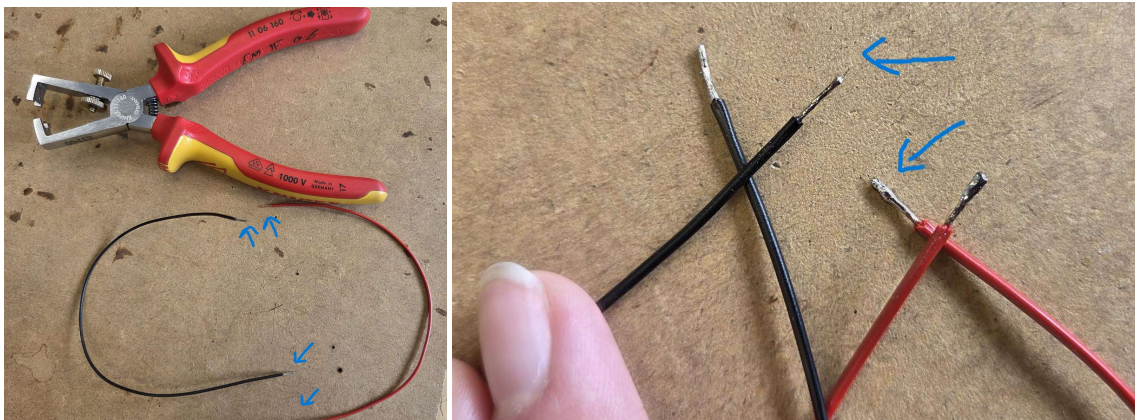
Step one: Drilling

To avoid damaging the station, the station must be first removed from the wooden housing. A hole was then drilled near the LED strip entry point to allow the cables to pass through.

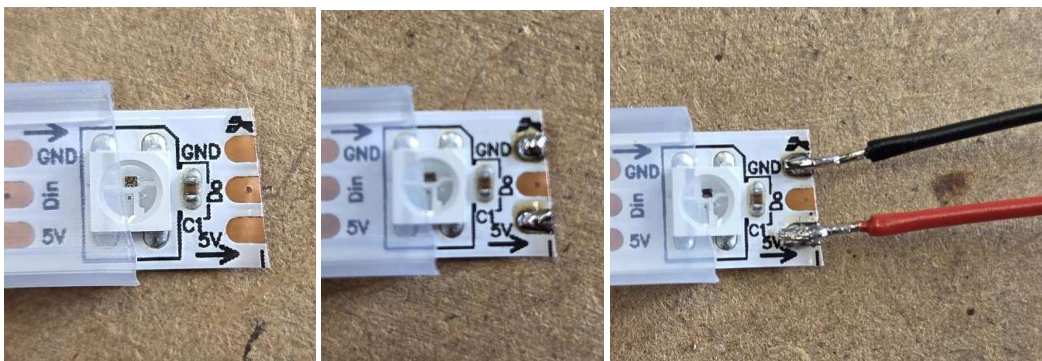


Step two: Preparing and soldering the wires

Using a wire stripper remove the insulation of the wires. Apply some solder to the exposed wire ends.

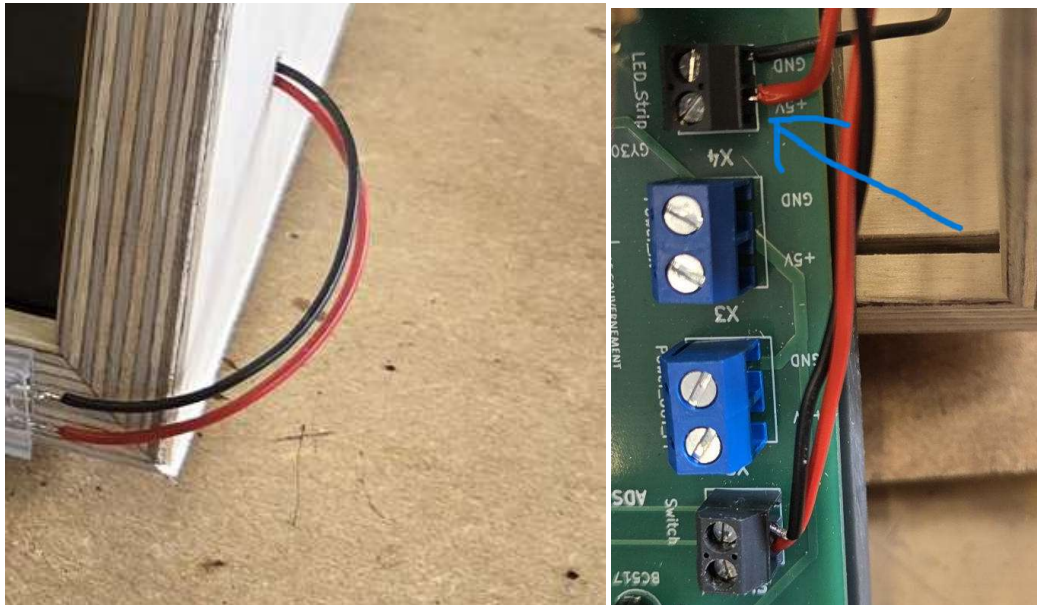


Then apply Solder on the GND and 5V pads, and the wires were soldered in place (black to GND, red to 5V).



Step three: Installing the LED strip

The wires were then passed through the drilled hole and connected to the GND and 5V terminals on the LED_Strip.



Lastly the LED strip was then glued to the front of the wooden housing using a glue gun.



Error:

No mistakes or obstacles have been observed in this sprint.